The present work was submitted to the Chair of Methods for Model-based Development in Computational Engineering

Hybrid Simulations with GNN Preconditioning for Flow Simulations

Hybrid-Simulationen mit GNN-Vorkonditionierung für Strömungssimulationen

Master's Thesis

Masterarbeit

presented by / von

Nain, Abhishek

Supervisor / Betreuer:

Ingo Steldermann

First Supervisor / Erstbetreuer:

Prof. Dr. Julia Kowalski

Aachen, *22 June 2026*

Abstract

A computational fluid dynamics (CFD) solver reaches its final flow solution iteratively, starting from an initial state and refining it until convergence. Because the computational effort depends on the number of iterations required, a better initial guess has the potential to reduce the solver work needed to reach the same solution. A trained machine-learning model can produce such a guess directly, without performing the iterative solution procedure, but its prediction is only approximate. Lacking the physical guarantees to satisfy the governing equations, it cannot reach strict convergence tolerances independently.

This thesis therefore couples the two rather than replacing the solver. A graph neural network (GNN) produces a fast initial guess, and OpenFOAM, a widely used open-source CFD solver, refines it to a converged numerical solution. A GNN suits this task because a simulation mesh naturally forms a graph of cells connected to their neighbours. The network is trained on 240 channel-flow cases.

Two inference strategies are tested. Injecting a fresh prediction at every step of a time-dependent (transient) simulation degrades performance, increasing the required solver iterations. However, using a learned field as the initial guess for a steady-state simulation reduces the solver work in every one of the 30 test cases, cutting total iterations by 11 to 18

The lasting outcome is a reusable OpenFOAM coupling interface, a single configurable hook into which other or improved models can be dropped, together with a clear account of when a learned initial guess helps an established solver and when it does not. Translating the demonstrated reductions in solver iterations into overall wall-clock speed-ups remains a focus for future work.

Table of contents

Abstract iii

List of Figures v

List of Tables vi

Abbreviations viii

1. Introduction 1

2. Background 4
2.1. Governing Equations . . . 4
2.2. Finite Volume Discretization . . . 5
2.3. Pressure-Velocity Coupling . . . 6
2.4. Iterative Linear Solvers . . . 8
2.5. Graph Neural Networks . . . 10
2.6. Related Work . . . 16

3. Methodology 18
3.1. Problem Setup . . . 18
3.2. Mesh Generation . . . 21
3.3. Dataset Construction . . . 21
3.4. Model Architecture . . . 23
3.5. Training Procedure . . . 25
3.6. Solver Coupling . . . 26
3.7. Benchmark Design . . . 32

4. Results 34
4.1. Training Results . . . 34
4.2. Solver-Coupled Benchmark Results . . . 38
4.3. Interpretation . . . 41

5. Conclusion 46
5.1. Summary . . . 46
5.2. Practical Implications . . . 47
5.3. Limitations . . . 47
5.4. Future Work . . . 48

References 50

Appendices **54**

A. Solver Coupling Configuration **54**

B. Current Benchmark Evidence **56**

C. Reproducibility and Artifact Map **57**

D. Aggregation Rules **60**

E. Case Parameter Distributions **61**

List of Figures

1.1. Predictor-corrector view of the coupling, where a learned prediction starts each solve close to the converged value and the corrector closes the leftover error. . . . 1
1.2. Three coupling modes: (a) the standard transient carry-forward, (b) a learned field injected at every step of a transient run, and (c) a learned field used once to initialize a steady solve. . . . 3

2.1. Finite-volume mesh connectivity represented as a graph. . . . 10
2.2. Node and edge feature vectors used by the GNN. . . . 12
2.3. One message-passing round for a single node. . . . 12
2.4. Finite-volume flux exchange (a) and the analogous graph message passing (b). . . . 13
2.5. MeshGraphNet encoder-processor-decoder with one of the K processor blocks expanded; symbols as in the equations above. . . . 15
2.6. Positioning of this thesis within ML-accelerated CFD literature. . . . 16

3.1. End-to-end workflow from case generation to OpenFOAM coupling. . . . 18
3.2. Example case-011 mesh, geometry labels, and boundary patches. . . . 20
3.3. The MGN of Figure 2.5 as instantiated in this work, with feature dimensions and solver-facing inputs and outputs. . . . 24
3.4. One injected time step: the hook gathers and packs the fields, the GPU evaluates the TorchScript module while the simulation process waits, and the blended prediction is written back before the time step is solved; the module is loaded once per case. . . . 28

4.1. Velocity magnitude for held-out case 188: undeveloped field, GNN warm start (100-step rollout), and converged reference. . . . 34
4.2. Training and validation loss over 100 epochs. . . . 35
4.3. Velocity (left) and pressure (right) RMSE against rollout step over 30 test trajectories, for the baseline and noise-trained models; shaded bands are the across-case standard deviation. . . . 37
4.4. OpenFOAM reference and baseline and noise-trained GNN rollouts (velocity magnitude) for case 188 at steps 100 and 200. . . . 37
4.5. Total- and pressure-iteration change relative to the cold baseline across the four model-assisted runs. . . . 40
4.6. One-step prediction-error power spectrum for u_x, u_y, p over the 30 test trajectories, normalized over wavenumbers $k \geq 1$ with the domain-mean ($k = 0$) component removed: (a) normalized spectra; (b) cumulative power fraction. . . . 42

E.1. Sampled case-parameter distributions across the 300-case family. . . . 61

List of Tables

3.1. Parameter ranges for the 300-case data-generation set. 19
3.2. Boundary conditions for the generated transient cases. 20
3.3. Fixed OpenFOAM solver settings for the transient data-generation cases and the steady warm-start cases. 20
3.4. Ten-dimensional node feature vector used by the encoder. 23
3.5. Mainline full-data checkpoint hyperparameters (LayerNorm in the encoder and processor MLPs, none in the decoder). 24

4.1. Pilot architecture-study results. 35
4.2. One-step RMSE on the test split. 36
4.3. 100-step rollout RMSE on fluid-only internal nodes. 36
4.4. Injection-path sanity controls on cases 011, 094, and 188; injection counts are totals over the three cases, the identity modes run to $t = 2.0\,\mathrm{s}$ and `bad_zero` to $t = 0.1\,\mathrm{s}$. 38
4.5. Repeated transient injection on the 30-case test split (`endTime=2.0 s`, injection at every time step). 39
4.6. Steady-state warm-start sweep across the 30-case test split. 39
4.7. Measured costs per panel, per case and cumulative over the 30 cases; each panel separates the simulation process's CPU work from the model-side wall-clock costs outside it. 44

B.1. Current benchmark evidence used for the thesis solver claims. 56

C.1. Main artifacts used by the thesis pipeline. 57
C.2. Main software dependencies used to produce the thesis results. 58
C.3. OpenFOAM build targets required for solver-coupled inference. 58

D.1. Sign convention used in all results tables. 60

Abbreviations

AMP Automatic Mixed Precision
CFD Computational Fluid Dynamics
FVM Finite Volume Method
GAMG Geometric-Algebraic Multigrid
GNN Graph Neural Network
MGN MeshGraphNet
MLP Multi-Layer Perceptron
MSE Mean Squared Error
PIMPLE merged PISO-SIMPLE
PISO Pressure-Implicit with Splitting of Operators
SIMPLE Semi-Implicit Method for Pressure-Linked Equations

1. Introduction

Computational Fluid Dynamics (CFD) is a core tool across aerospace, automotive, and process engineering, yet the computational cost of high-fidelity simulations remains a persistent bottleneck [1]. After discretization of the incompressible Navier-Stokes equations, for example with the Finite Volume Method (FVM), the governing physics reduce to a sequence of large, sparse linear systems that must be solved to convergence at every time step or outer iteration [2]. For incompressible flows, this linear-solver work is dominated by the pressure-correction stage, the step that enforces mass conservation by projecting the velocity field onto a divergence-free subspace. This step produces a Poisson-type system whose size scales with the mesh. The number of iterations required to solve each such system depends directly on the quality of the initial guess supplied to the iterative solver [3]; a guess already close to the solution reduces the residual the solver must eliminate, lowering the iteration count. Standard solvers rely on basic heuristics for this guess, such as carrying forward the previous time step's field or using a uniform constant, leaving room for a trained model to supply a much closer starting point. This dependence is the central lever this thesis examines (Figure 1.1).

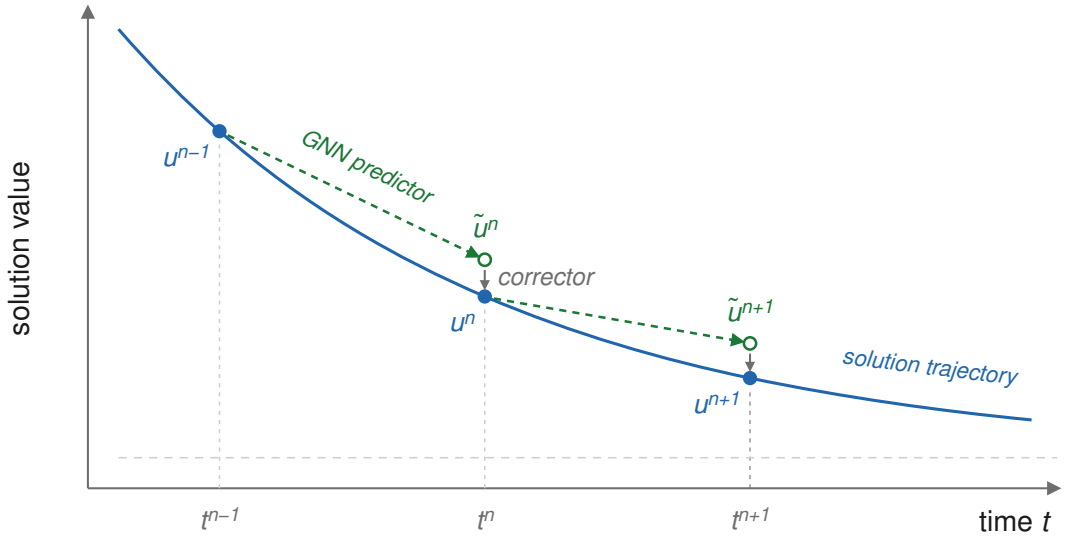


Figure 1.1.: Predictor-corrector view of the coupling, where a learned prediction starts each solve close to the converged value and the corrector closes the leftover error.

A prominent trend in learned CFD acceleration is the full neural surrogate, a model trained to step the flow field forward, prediction after prediction, in place of the numerical solver altogether [4], [5], [6]. While such models can be computationally inexpensive

to evaluate, they do not inherently enforce the discrete governing equations, and their predictive accuracy may decrease when applied outside the conditions represented in the training data. A complementary line of work retains the validated numerical solver and uses the learned model only to inform or accelerate it [7], [8]; the model supplies a prior estimate while the solver enforces consistency with the discretized physics to the required tolerance. This thesis follows that paradigm, using OpenFOAM as the numerical solver. The hybrid approach functions as a predictor-corrector scheme, which Figure 1.1 illustrates along the solution trajectory. The trained Graph Neural Network (GNN) acts as the explicit predictor, proposing an intermediate flow field, while OpenFOAM acts as the implicit corrector, eliminating the leftover error until the field satisfies the governing physics.

Producing such a guess requires predicting the pressure and velocity fields over the solver's unstructured mesh. Finite-volume meshes have arbitrary cell connectivity. Cell count, shape, and neighbourhood topology vary across the domain and across problem instances, with no regular spatial layout. GNNs are well suited to this setting because they operate on graph-structured data of arbitrary topology without requiring a fixed grid [9]. The mapping from mesh to graph is direct. Each cell becomes a node and each internal face between two cells becomes an edge, so the GNN acts on the same neighbourhood structure the solver already assembles, with no resampling or interpolation that a convolutional architecture would require. This local connectivity also mirrors the stencil coupling inherent in the FVM, making mesh-based learned simulation a natural fit for this data structure [4].

The central question is therefore: *Can a trained GNN, coupled to OpenFOAM as a learned initial-guess provider, reduce the linear-solver work needed to reach the same residual tolerances, without changing the solver itself?*

To answer this, the GNN predictor can be coupled to the solver using two distinct strategies (Figure 1.2 contrasts both with the standard transient carry-forward). In a time-dependent simulation, the coupling forms a repeated predictor-corrector cycle; a new prediction is injected at every time step to start the outer iteration loop. Alternatively, in a steady-state simulation, it acts as a single global prediction, with the field written just once to initialize the entire run (a *warm start*). Because a steady solve normally starts from a crude uniform field, it provides the clearest opportunity for a trained model to supply a large initial correction. The evaluation of these two inference modes reveals opposite results. Injecting a prediction at every time step of a transient run degrades performance, leading the solver to require more iterations to converge. In contrast, when used to initialize a steady-state solve, the single injection reduces the iterations required for convergence in all 30 test cases. The current implementation is not yet a complete wall-clock accelerator, but the iteration results identify a clear role for learned fields as one-time warm starts for steady OpenFOAM solves.

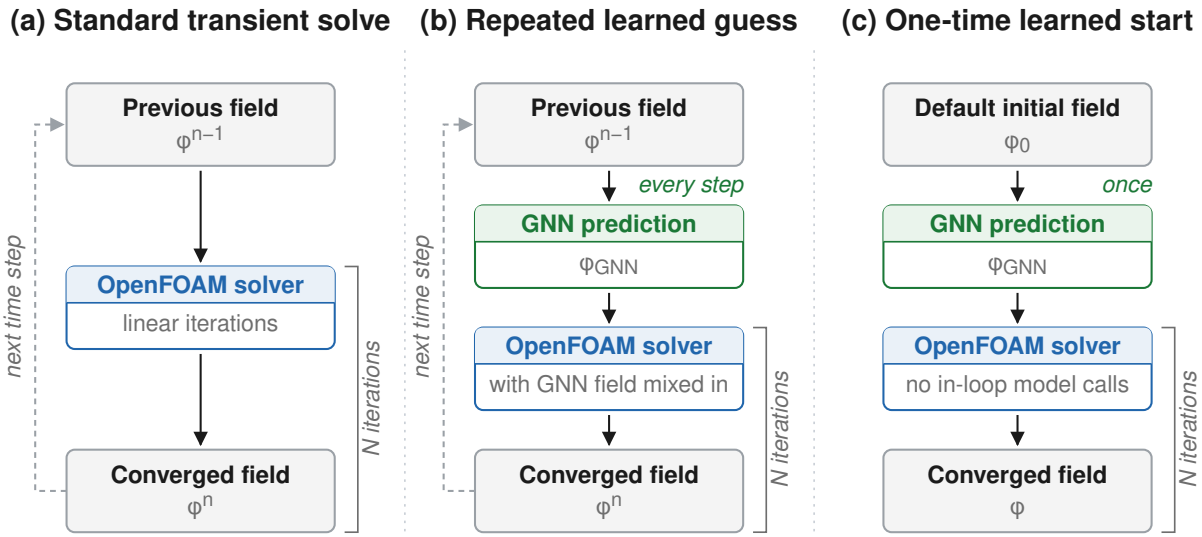


Figure 1.2.: Three coupling modes: (a) the standard transient carry-forward, (b) a learned field injected at every step of a transient run, and (c) a learned field used once to initialize a steady solve.

To study this behaviour, the GNN is trained on 240 cases from a family of 300 two-dimensional channel-flow geometries. The complete pipeline, from parametric case generation through model training to solver-coupled inference, is implemented and tested for OpenFOAM 12. Two control experiments separate the effect of the coupling from that of the learned field. Feeding the solver's own field back through the interface leaves the iteration count unchanged, while feeding a deliberately uninformative field increases it. This confirms the coupling itself is neutral and the solver responds to the quality of the injected field.

Chapter 2 reviews the numerical background (incompressible flow, the finite-volume method, pressure-velocity coupling, and iterative linear solvers) and introduces GNNs and the related machine-learning literature. Chapter 3 describes the full pipeline: case generation, the dataset and graph representation, the model architecture and training, the OpenFOAM coupling, and the benchmark design. Chapter 4 reports the training behaviour and the solver-coupled benchmark results, then interprets them and compares them with related work. Chapter 5 summarizes the findings, states the limitations, and outlines directions for future work.

2. Background

This chapter reviews the concepts needed for the solver-coupled study. It begins with incompressible flow, finite-volume discretization, pressure-velocity coupling, and iterative linear solvers, because these determine how OpenFOAM responds to a changed initial field. It then gives a compact introduction to graph neural networks and summarizes the closest machine-learning work, with emphasis on learned initialization rather than on learned operators applied inside the linear solver itself.

2.1. Governing Equations

2.1.1. Incompressible Navier-Stokes Equations

The motion of an incompressible Newtonian fluid is governed by the Navier-Stokes equations, consisting of the continuity equation (mass conservation) and the momentum equation:

$$\nabla \cdot \mathbf{u} = 0 \tag{2.1}$$

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u}\mathbf{u}) = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u} \tag{2.2}$$

where $\mathbf{u}$ is the velocity field, p is the pressure, ρ is the fluid density, and ν is the kinematic viscosity. The continuity equation (Equation 2.1) enforces the incompressibility constraint that the velocity field must be divergence-free everywhere in the domain. The momentum equation (Equation 2.2) balances the rate of change of momentum against convective transport $\nabla \cdot (\mathbf{u}\mathbf{u})$, the pressure gradient ∇p, and viscous diffusion $\nu \nabla^2 \mathbf{u}$.

2.2. Finite Volume Discretization

2.2.1. From PDEs to Algebraic Systems

The FVM discretizes the governing equations by integrating over control volumes (cells) that partition the computational domain. For a generic transport variable ϕ, the integral form over a control volume V_P with boundary ∂V_P reads:

$$\frac{\partial}{\partial t}\int_{V_P}\phi\,dV + \oint_{\partial V_P}\phi\,\mathbf{u}\cdot d\mathbf{S} = \oint_{\partial V_P}\Gamma\nabla\phi\cdot d\mathbf{S} + \int_{V_P}S_\phi\,dV \tag{2.3}$$

where $\mathbf{S}$ is the outward-pointing surface area vector, Γ is the diffusion coefficient, and S_ϕ is a volumetric source term. The source gathers everything not carried by the convective or diffusive face fluxes; for the momentum equation it is the pressure gradient together with any body force such as gravity, and for a passive scalar it is zero. The key step is to approximate the surface integrals as sums over the faces of each cell. For a cell P with neighbours N_f sharing face f:

$$\sum_f \phi_f\,(\mathbf{u}\cdot\mathbf{S})_f \approx \sum_f \phi_f\,F_f, \qquad F_f = \mathbf{u}_f\cdot\mathbf{S}_f \tag{2.4}$$

where F_f is the volumetric face flux and ϕ_f is the face value of ϕ, obtained by interpolation from the adjacent cell-centre values. The choice of interpolation scheme determines the numerical properties of the discretization. Central differencing provides second-order accuracy but can produce oscillations in convection-dominated regions, while upwind differencing is first-order but unconditionally bounded. OpenFOAM provides a family of schemes (linear, upwind, linearUpwind, limitedLinear) that balance accuracy and stability [2].

2.2.2. Assembly into Linear Systems

After discretization, the transport equation for ϕ at each cell P takes the form:

$$a_P\phi_P = \sum_N a_N\phi_N + b_P \tag{2.5}$$

where a_P is the central coefficient, a_N are the neighbour coefficients (one per face), and b_P collects the discretized source term S_ϕ together with the boundary contributions. Collecting these equations over all cells yields a sparse linear system $A\mathbf{x} = \mathbf{b}$, where the sparsity pattern of A mirrors the mesh connectivity, with cell P coupled only to its immediate face-neighbours. In OpenFOAM, this sparse matrix is stored in the

`lduMatrix` format, a compact representation that stores only the diagonal, the upper triangle, and the lower triangle, exploiting the symmetric sparsity pattern of the mesh topology [10].

This assembled system is the object the rest of this work acts on. OpenFOAM does not invert A directly; it solves $A\mathbf{x} = \mathbf{b}$ iteratively (Section 2.4), starting from an initial guess $\mathbf{x}_0$ (in a transient run, the previous time step's converged field) and driving the residual $\mathbf{r} = \mathbf{b} - A\mathbf{x}$ below the solver tolerance. How many iterations this takes depends on how close $\mathbf{x}_0$ already is to the solution, which is exactly the quantity a learned initial guess sets out to improve. The same neighbour-only sparsity that makes A compact also defines a graph, one node per cell and one edge per internal face, and it is this graph that the network in Section 2.5 operates on, so that its local updates coincide with the face-neighbour coupling of the discretization.

2.3. Pressure-Velocity Coupling

2.3.1. The Coupling Problem

The incompressible Navier-Stokes equations present a fundamental numerical difficulty, since the continuity equation (Equation 2.1) contains no time derivative of pressure, so the pressure field cannot be advanced in time directly. Instead, the pressure must be determined at each time step such that the resulting velocity field satisfies the divergence-free constraint. This is the *pressure-velocity coupling problem*, and all incompressible flow solvers must address it through some form of segregated or coupled solution strategy [1].

2.3.2. The SIMPLE Algorithm

The Semi-Implicit Method for Pressure-Linked Equations (SIMPLE) algorithm [11] addresses the coupling problem through a predictor-corrector approach. Given a current estimate of the pressure field p^*:

1. **Momentum predictor.** Solve the discretized momentum equation with p^* to obtain an intermediate velocity $\mathbf{u}^*$ that satisfies momentum but not continuity.
2. **Pressure correction.** Derive a pressure-correction equation by substituting the momentum equation into the continuity constraint. Solving this Poisson-like equation yields a correction p' such that the updated velocity $\mathbf{u}^{**} = \mathbf{u}^* - \frac{V_P}{a_P}\nabla p'$ satisfies continuity.
3. **Field update.** Update the pressure $p = p^* + \alpha_p\, p'$ and velocity fields, where α_p is an under-relaxation factor that stabilizes the iteration.

Steps 1–3 are repeated until the pressure and velocity fields converge. Under-relaxation is essential because the pressure correction equation is derived under simplifying assumptions (neglecting neighbour corrections), and full updates would diverge.

2.3.3. The PISO Extension

The Pressure-Implicit with Splitting of Operators (PISO) algorithm [12] extends SIMPLE by performing multiple pressure-correction steps within each time step, without re-solving the momentum equation. This eliminates the need for under-relaxation and makes the algorithm more suitable for transient simulations with small time steps. In practice, 2–3 corrector steps are sufficient to drive the continuity error well below the solver tolerance.

2.3.4. The PIMPLE Algorithm

The merged PISO-SIMPLE (PIMPLE) algorithm, used throughout this work, combines the two, wrapping the PISO inner correctors inside SIMPLE-style outer loops. Making the nesting explicit, each time step advances as:

```
for each time step t^n -> t^{n+1}:
    for k = 1 ... n_outer:                  # outer (SIMPLE) correctors
        solve the momentum predictor -> u*    # using the latest pressure
        for m = 1 ... n_inner:                # inner (PISO) pressure
        ↪  correctors
            solve the pressure-correction equation -> p'
            correct the face fluxes and velocity -> u   # now divergence-free
        under-relax p and u                   # outer loop only
```

The outer loop plays the role of SIMPLE and may under-relax, while the inner loop is PISO, refining the pressure-velocity coupling without re-solving momentum. The specific corrector counts used for the generated cases are given in Section 3.1.

The pressure equation dominates the computational cost of each time step. Unlike the velocity equations, which produce well-conditioned diagonally dominant systems that converge in a few inexpensive iterations, the pressure Poisson equation is an elliptic system that requires global information propagation to enforce the incompressibility constraint. The condition number of the pressure matrix grows with the number of cells, making iterative convergence slower on finer meshes. This asymmetry (cheap velocity solves, expensive pressure solves) is a fundamental characteristic of segregated incompressible flow solvers [1].

2.4. Iterative Linear Solvers

2.4.1. Why Iterative Methods

The discretized equations produce sparse linear systems $A\mathbf{x} = \mathbf{b}$ at each corrector step. For the mesh sizes encountered in practical CFD (thousands to millions of cells), direct factorization methods are prohibitively expensive in both memory ($O(n^2)$ fill-in for general sparse matrices, where n is the number of cells, i.e. the dimension of A) and compute ($O(n^3)$ for dense factors). Iterative methods, by contrast, require only $O(\text{nnz})$ storage and $O(\text{nnz})$ work per iteration, where nnz is the number of non-zero entries in A, making them the standard choice for CFD applications [3].

2.4.2. Krylov Methods and Preconditioning

Krylov methods construct approximate solutions from repeated applications of the sparse matrix to the residual. For symmetric positive-definite systems such as the pressure equation, conjugate-gradient methods minimize the error in an energy norm [3]. In practice, these methods are almost always used with preconditioning, because the number of iterations depends strongly on the conditioning of the matrix. A preconditioner applies an approximate inverse M^{-1} that transforms the system into an equivalent form that is easier for the iterative method to solve.

In the OpenFOAM cases used here, pressure is solved with Geometric-Algebraic Multigrid (GAMG), while velocity is solved with `smoothSolver` and symmetric Gauss-Seidel smoothing. The thesis therefore focuses less on Krylov theory itself and more on the quantity that the learned model can influence directly, the initial field from which these linear solves start. In this broad, solver-coupling sense, the *preconditioning* named in the title of this thesis refers to that learned starting field, an improved initial field for the outer pressure-velocity iteration, rather than the strict M^{-1} operator defined above; the assembled linear systems, residual tolerances, and OpenFOAM linear solvers all remain unchanged.

2.4.3. Multigrid Methods

Multigrid methods address the fundamental limitation of a *smoother*. A smoother is a simple iterative method, such as Gauss-Seidel, that cheaply damps high-frequency error (the part that varies on the scale of the mesh spacing) but is slow to remove the low-frequency error that spans many cells. Multigrid overcomes this by restricting the residual to a sequence of progressively coarser grids, where the low-frequency error becomes high-frequency and is efficiently smoothed, then prolongating the correction back to the fine grid [13].

OpenFOAM's GAMG solver implements an algebraic multigrid strategy, constructing the coarse levels by agglomerating cells based on the matrix coefficients, without requiring a geometric hierarchy of meshes [14]. Each V-cycle alternates smoothing on the current level with restriction of the residual to coarser levels and prolongation of the corrections back. For the pressure equation it is paired with Gauss-Seidel smoothing in this work.

2.4.4. The Role of the Initial Guess

Every iterative solver begins from an initial guess $\mathbf{x}_0$ and reduces the residual $\|\mathbf{b} - A\mathbf{x}_k\|$ toward zero. The number of iterations required to reach a target tolerance depends directly on the initial residual $\|\mathbf{r}_0\| = \|\mathbf{b} - A\mathbf{x}_0\|$; a better initial guess yields a smaller $\|\mathbf{r}_0\|$ and, all else being equal, fewer iterations to converge.

What the guess must be close to differs between the two solver settings, even though the mechanics are the same. Discretizing the momentum equation (Equation 2.2) implicitly in time replaces the time derivative $\partial\mathbf{u}/\partial t$ with the difference $(\mathbf{u}^{n+1} - \mathbf{u}^n)/\Delta t$, adding a $1/\Delta t$ term to the diagonal of A and the known $\mathbf{u}^n/\Delta t$ to $\mathbf{b}$; the assembled per-step system is again $A\mathbf{x} = \mathbf{b}$, with the same initial residual $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ as the steady case [3]. That time-derivative term anchors the system's solution one small step ahead of the previous field, so the standard transient guess is the previous time step's solution carried forward, and when the flow evolves slowly relative to Δt this guess is already close and the solver converges quickly. A steady solve assembles the same kind of system, but its solution is the developed flow, and the run normally starts from a uniform field, leaving a far larger gap between guess and solution.

A learned field reduces iterations exactly when it lands closer to that system's solution than the default guess does. In a transient run this is a conservative target, namely predicting the next step better than the carry-forward while preserving the physical trajectory; the field cannot be used to steer the run toward a guessed end state. In a steady solve there is no trajectory to preserve (the iteration index is a pseudo-time marching the system toward its fixed point), so a learned field is free to sit as close to the converged state as it can, which is exactly what is wanted [7]. In both settings the solver still iterates to its unchanged tolerances and guarantees the same converged solution; the guess affects *how many iterations* are needed, not the final answer. The two roles are evaluated separately: a per-step injection inside a transient run, and a single learned field used to initialize a steady solve.

The initial guess matters most for the pressure equation, since the pressure correction communicates information globally (Section 2.3); a pressure field that already contains the right large-scale structure reduces the number of GAMG cycles OpenFOAM needs.

2.5. Graph Neural Networks

The solver works by repeatedly solving a sparse linear system on the mesh graph, and the closer the starting field is to the solution, the fewer iterations each solve needs. This thesis predicts that starting field with a graph neural network, a model that operates directly on that graph.

2.5.1. Graphs as a Representation for Computational Meshes

A computational mesh naturally defines a graph. Each cell is a node, and each internal face connecting two cells defines an edge between the corresponding nodes. Figure 2.1 illustrates this correspondence for a simple six-cell mesh. The resulting graph carries exactly the connectivity information that the FVM uses to assemble its linear systems. The sparsity pattern of the matrix A in Equation 2.5 is the adjacency matrix of the mesh graph. For each internal face, two directed edges are created (one in each direction), so that information can flow both ways between neighbouring cells when the network updates them.

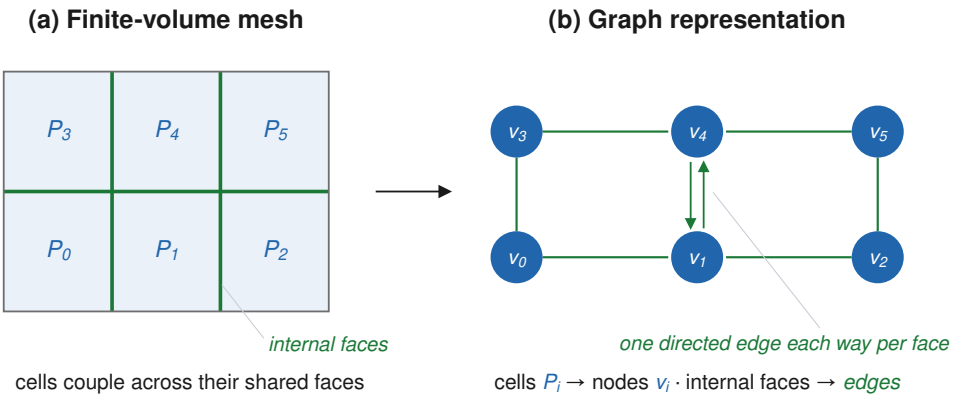


Figure 2.1.: Finite-volume mesh connectivity represented as a graph.

This correspondence makes GNNs a natural architecture for learning on mesh-based data. Unlike convolutional neural networks, which require a regular grid structure, GNNs operate directly on arbitrary graph topologies and are therefore applicable to the unstructured meshes used in practical CFD [9]. A GNN also shares the locality of the finite-volume method, updating each cell mainly from its immediate neighbours, just as a finite-volume update depends mainly on the neighbouring cells.

2.5.2. Node and Edge Features

The mesh-to-graph correspondence also fixes what each node and edge carries. A node represents a finite-volume cell and holds a feature vector whose entries (its *channels*) are

of two kinds. The *dynamic* channels are the quantities that change as the flow evolves (the two in-plane velocity components and the gauge pressure, the pressure with its domain-mean subtracted) and give the cell's current state. The *static* channels are fixed for a given case (the cell volume, a boundary-type indicator, the case inlet velocity, and the cell-centre position) and let the model condition its prediction on the geometry and on the boundary role of each cell, so that a near-wall cell and an interior cell are treated differently even when their instantaneous states look alike.

An edge represents an internal face between two cells and carries the *relative* geometry of that connection, the displacement between the two cell centres and its length. These are exactly the geometric ingredients the FVM uses when it interpolates face values and evaluates gradients between neighbours, so each message has access to the same local geometry that enters the discrete operators.

Of the static channels, the cell-centre position is the only one not used by the discrete operators, which rely on the relative geometry carried by the edges. It is included because one model is applied to every case, and the cases differ in their geometry; the step height and channel length vary, and each case is meshed separately. The model predicts a cell's update from a fixed-size neighbourhood, but the correct update depends on where the cell sits in its particular domain, which differs from case to case. Two cells with identical local states and stencils, drawn from differently proportioned cases, can therefore require different updates, and message passing on a bounded neighbourhood cannot by itself recover where a cell sits in its domain. The absolute position supplies that location directly. Removing this feature in a control clearly worsens the model, confirming that it carries information the local neighbourhood cannot reconstruct on its own.

The exact feature layout, including normalization, is given in Section 3.3; Figure 2.2 summarizes the two vectors, with ten channels per node (three dynamic, seven static) and three per edge.

2.5.3. Message Passing Framework

A graph neural network operates on a graph $G = (V, E)$, a set of nodes (vertices) V and a set of edges E, where each node v carries a feature vector and each edge (u, v) carries geometric features $\mathbf{e}_{uv}$. Most graph-convolutional layers, including the GCN [15], GraphSAGE [16], and graph attention networks [17], are instances of a common *message passing* framework [18] that updates the node features over K rounds of local communication. Writing $\mathbf{h}_v^{(k)}$ for the latent feature vector of node v after round k, initialized at $k = 0$ from the node's input features, one round is

$$\mathbf{m}_v^{(k)} = \bigoplus_{u \in \mathcal{N}(v)} \phi^{(k)}\left(\mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)}, \mathbf{e}_{uv}\right), \qquad \mathbf{h}_v^{(k)} = \psi^{(k)}\left(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(k)}\right) \tag{2.6}$$

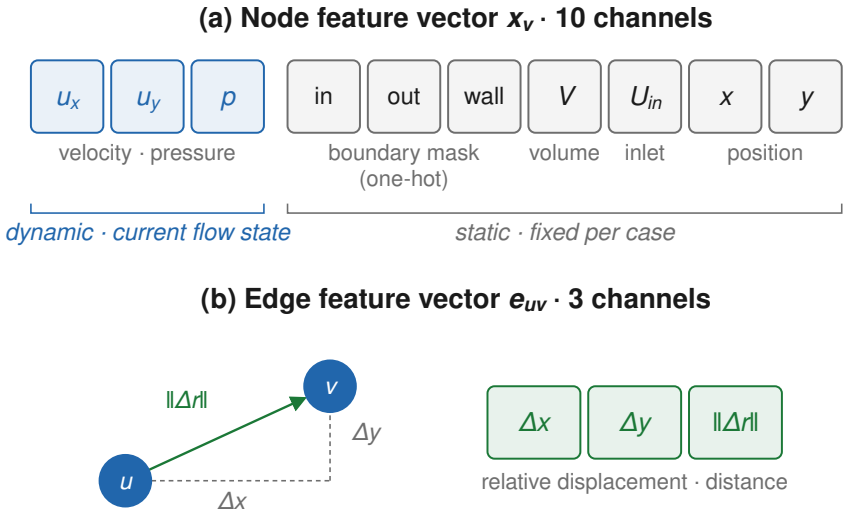


Figure 2.2.: Node and edge feature vectors used by the GNN.

Here $\mathcal{N}(v)$ is the set of neighbours of v (the cells sharing a face with it) and u ranges over them; $\phi^{(k)}$ is the *message function*, which builds a message from the sender state $\mathbf{h}_u^{(k-1)}$, the receiver state $\mathbf{h}_v^{(k-1)}$, and the connecting edge $\mathbf{e}_{uv}$; $\bigoplus$ is a permutation-invariant aggregation that combines the incoming messages into one vector $\mathbf{m}_v^{(k)}$ (concretely a sum, for the reason given in the next subsection); and $\psi^{(k)}$ is the *update function* that produces the node's new state. Both $\phi^{(k)}$ and $\psi^{(k)}$ are small neural networks, learned and shared across all nodes. After K rounds, each node has gathered information from its K-hop neighbourhood.

Figure 2.3 shows a single round of this process for one node.

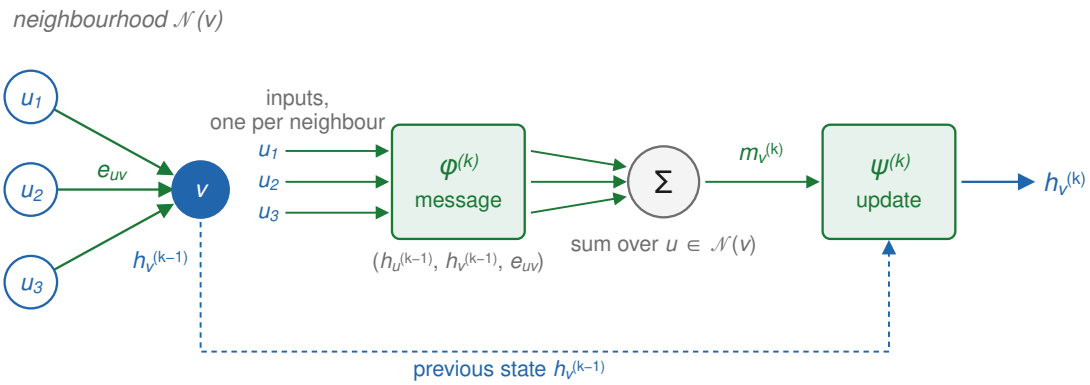


Figure 2.3.: One message-passing round for a single node.

2.5.4. Message Passing as Local Coupling

The analogy to the finite-volume method can be made more precise. A single message-passing round lets every cell gather information from its immediate face-neighbours and update its own latent state. This is structurally the same locality that a classical iterative solver exploits; a Gauss-Seidel or Jacobi sweep, and the flux and pressure-correction updates inside PIMPLE, also move information one cell at a time across the face connections. Stacking K rounds therefore gives each node a receptive field of K mesh hops, in the same way that information physically travels a limited number of cells per solver sweep; unrolled, the K rounds form a depth-K computation tree rooted at the node and spanning that neighbourhood. The pressure field is the difficult case, since enforcing incompressibility couples the entire domain at once (Section 2.3), and a model with a fixed number of message-passing steps cannot propagate information across the whole mesh in a single forward pass [19].

The aggregation $\bigoplus$ is chosen to be permutation invariant because a cell has no intrinsic ordering of its faces, so the prediction must not depend on how the neighbours happen to be indexed. Sum aggregation is used here because the combined neighbour contribution then scales with the number and strength of the connections, mirroring how a finite-volume balance sums fluxes over all faces. Among the simple aggregators, sum is also the most expressive [20].

Figure 2.4 places the two updates side by side, with a finite-volume cell collecting fluxes across its faces and a graph node collecting messages along its edges, each using only its immediate neighbourhood.

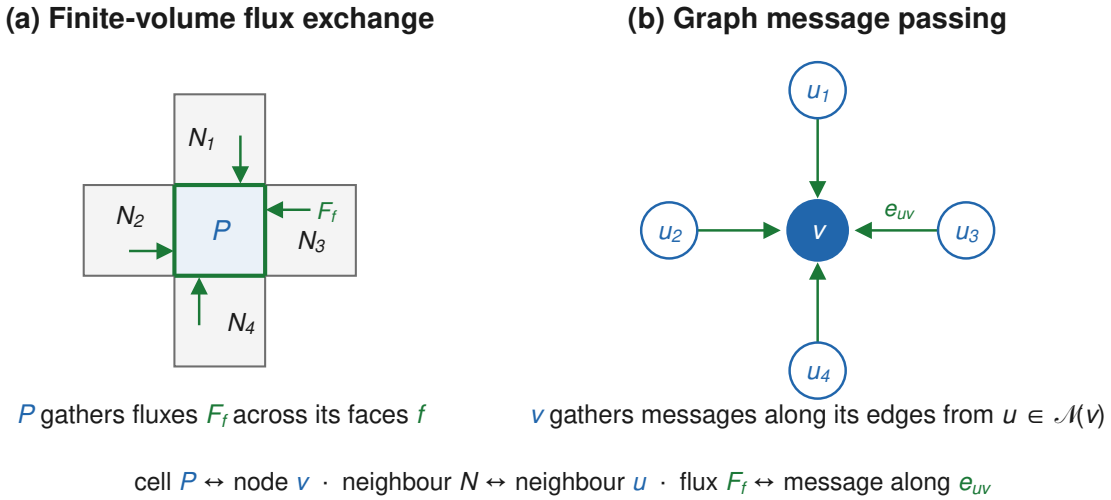


Figure 2.4.: Finite-volume flux exchange (a) and the analogous graph message passing (b).

2.5.5. MeshGraphNet Architecture

The MeshGraphNet (MGN) architecture [4] specializes this framework for mesh-based simulation with an encoder-processor-decoder structure. Each learnable function in it (the message and update functions above, and the encoder and decoder introduced here) is a *multilayer perceptron* (MLP), a small stack of linear layers separated by nonlinearities and the standard trainable building block. The encoder maps the raw node and edge feature vectors $\mathbf{x}_v$ and $\mathbf{x}_{uv}$ (the node and edge channels defined above) into a common latent space,

$$\mathbf{h}_v^{(0)} = \mathrm{MLP}_{\mathrm{node}}(\mathbf{x}_v), \quad \mathbf{e}_{uv}^{(0)} = \mathrm{MLP}_{\mathrm{edge}}(\mathbf{x}_{uv}),$$

so that message passing acts on learned representations rather than on the raw physical quantities.

The processor is the sequence of K message-passing blocks. Each block is the concrete MGN form of Equation 2.6, updating first the edge and then the node state, each with a skip connection (also called a residual connection) that adds the update to the previous state,

$$\mathbf{e}_{uv}^{(k)} = \mathbf{e}_{uv}^{(k-1)} + \mathrm{MLP}_e^{(k)}\left([\mathbf{e}_{uv}^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)}]\right)$$

$$\mathbf{h}_v^{(k)} = \mathbf{h}_v^{(k-1)} + \mathrm{MLP}_n^{(k)}\left([\mathbf{h}_v^{(k-1)}, \sum_{u \in \mathcal{N}(v)} \mathbf{e}_{uv}^{(k)}]\right)$$

where $[\cdot, \cdot]$ denotes concatenation and the sum is the permutation-invariant aggregation introduced earlier. The skip connections and layer normalization (LayerNorm) [21] keep the stacked blocks trainable; this processor is where the local neighbourhood coupling of the previous subsections actually happens.

The decoder is a final MLP that reads the processed node states out to the target, which in this thesis is the per-node field increment:

$$\hat{\mathbf{y}}_v = \mathrm{MLP}_{\mathrm{dec}}(\mathbf{h}_v^{(K)})$$

Figure 2.5 assembles the three stages, with one processor block expanded to show the wiring of the update equations, namely the node states feeding the edge update, the aggregation over $\mathcal{N}(v)$ feeding the node update, and the two skip additions.

The whole network is trained end-to-end to minimize the Mean Squared Error (MSE) between predicted and ground-truth increments. The number of message-passing

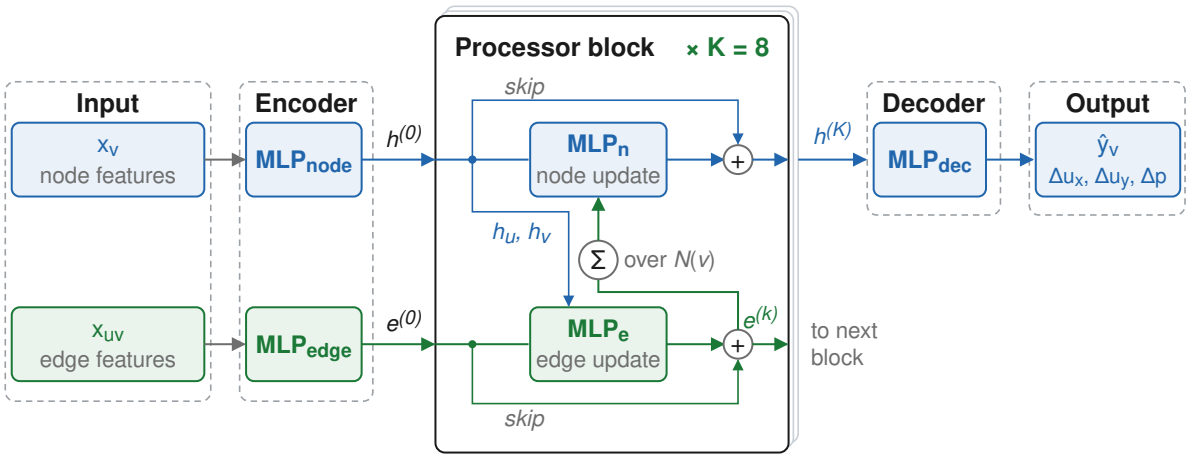


Figure 2.5.: MeshGraphNet encoder-processor-decoder with one of the K processor blocks expanded; symbols as in the equations above.

steps K is the main cost lever, since each step is a full pass over all edges; it also fixes the receptive field discussed above. The mainline uses $K = 8$ as a receptive-field-versus-cost compromise; the effect of varying it is examined in the architecture study of Section 4.1. Very deep message passing can additionally over-smooth node representations [22], but that regime lies well beyond the depths used here.

2.5.6. Predicting Increments and Warm-Start Fields

Two modelling choices follow from the solver-coupled goal. First, the network predicts per-node field *increments* $(\Delta u_x, \Delta u_y, \Delta p)$ rather than absolute fields, since an increment varies less across the domain than the field itself, which makes it an easier regression target. These increment targets are additionally *normalized*, shifted and scaled to roughly zero mean and unit variance using statistics gathered over the training set, so the network learns on a common, well-conditioned scale, as is standard in learned simulators [4]. At inference the prediction is denormalized with the same statistics and added to the current state to recover the absolute field. Second, and more fundamentally, the model is used to produce an *initial field* for the solver rather than the final solution. The reasons are direct consequences of the previous subsections. The fixed message-passing depth discussed above means the network cannot exactly enforce the global incompressibility constraint that the pressure equation imposes. A one-step prediction with low point-wise error may still violate the discrete continuity and pressure-correction equations that the following linear solve enforces. And keeping OpenFOAM responsible for the final solve preserves its convergence guarantees (Section 2.4.4). These properties favour using the learned field as a warm start for a steady solve over a correction applied at every time step; both roles are nonetheless evaluated (Chapter 3, Chapter 4). Because each prediction advances the field by only a single step, the warm-start field for a steady solve is produced by applying the model

repeatedly, feeding each output back as the next input, an autoregressive procedure called a *rollout*, where a longer rollout advances the flow further before the solver takes over. The two regimes are made precise in Chapter 3 and compared in Chapter 4.

2.6. Related Work

Machine-learning methods for CFD can be placed on a spectrum by how much authority remains with the numerical solver after the network makes its prediction; Figure 2.6 organizes the field into four categories, from a model that replaces the solve to one that only assists a single linear system. This thesis sits in the solver-retained initial-guess category.

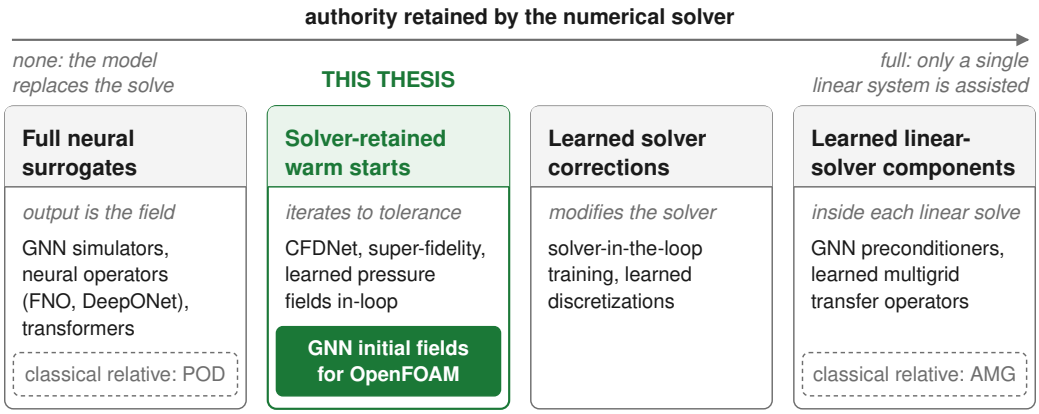


Figure 2.6.: Positioning of this thesis within ML-accelerated CFD literature.

Full neural surrogates replace the solver outright, so the model's output *is* the predicted field. GNN simulators roll out mesh or particle dynamics step by step [4], [5], message-passing PDE solvers learn the update rule directly on the graph [23], and *neural operators* such as the Fourier neural operator (FNO) [24] and DeepONet [25] learn a mapping between entire input and solution fields, predicting the solution in one shot. Recent transformer-based neural surrogates scale this approach to industrial automotive aerodynamics, predicting surface and volume fields directly from the geometry in seconds [26]. Their classical counterpart is reduced-order modelling such as proper orthogonal decomposition (POD), which projects the solution onto a few precomputed modes. None of these guarantee that the discrete equations are satisfied, the property this thesis keeps by retaining the solver.

Solver-retained initial guesses, where this thesis sits, keep the full solver and only hand it a better starting field. CFDNet lets the solver take a few warmup iterations, predicts the steady state from the intermediate field, and returns that prediction to the solver for refinement to the original convergence constraints [7]; super-fidelity methods start

from a cheap low-fidelity solution [27]; and others inject learned pressure fields into the pressure solve [28], [29], an idea going back to convolutional pressure-projection solvers [30], [31]. The solver still iterates to tolerance, so accuracy is unchanged and only the iteration count moves.

Learned solver corrections reach inside the algorithm. Solver-in-the-loop training learns a correction by back-propagating through a differentiable solver [8], and learned-discretization methods replace numerical stencils with trained ones inside a differentiable Navier-Stokes solver [32], sometimes constrained to preserve formal convergence [33]. These modify the solver itself; this thesis leaves OpenFOAM's algorithm untouched.

Learned linear-solver components act at the innermost level, inside each linear solve. Graph neural preconditioners [34] and learned multigrid transfer operators [35], [36] replace pieces of the linear solver. Their classical counterpart is algebraic multigrid (AMG) [14], which builds the same coarse-grid hierarchy analytically. This thesis instead leaves the OpenFOAM linear solvers unchanged and improves only the field-level initial guess.

3. Methodology

This chapter describes the complete pipeline built to couple a GNN to OpenFOAM as a learned initial-guess provider, from parametric case generation through model training to solver-coupled inference. Figure 3.1 shows its five stages, each detailed in the sections that follow; the implementation targets OpenFOAM 12 on a Linux HPC cluster with NVIDIA GPU support.

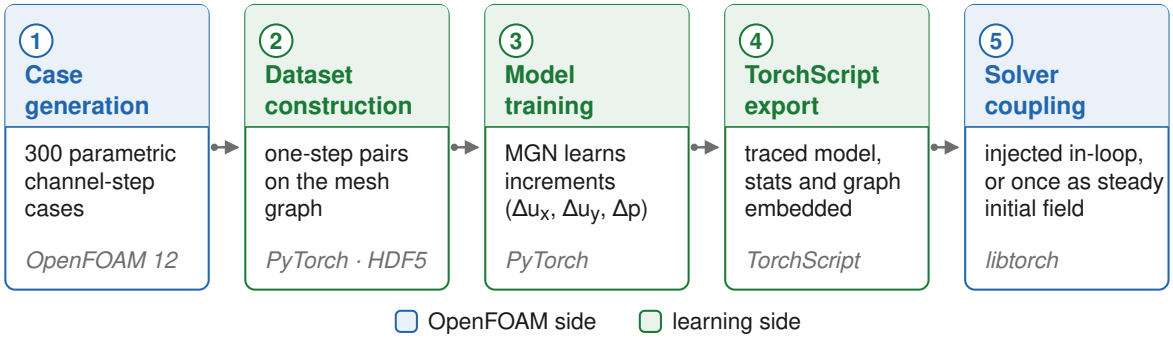


Figure 3.1.: End-to-end workflow from case generation to OpenFOAM coupling.

3.1. Problem Setup

The benchmark family is a two-dimensional incompressible internal-flow channel with a rectangular lower-wall step. The fluid domain is the L-shaped region obtained from

$$0 \leq x \leq x_{\max}, \qquad 0 \leq y \leq y_{\max}$$

after removing the solid step block

$$0 \leq x \leq x_{\mathrm{mid}}, \qquad 0 \leq y \leq y_{\mathrm{mid}}.$$

The inlet velocity U_{inlet} is prescribed as a uniform horizontal profile (a Dirichlet condition on velocity together with a zero normal pressure gradient), while the outlet uses the standard OpenFOAM outflow treatment, with a zero normal velocity gradient and the gauge pressure fixed to zero. The upper wall, lower wall, and step surfaces are

treated as fixed no-slip walls. These conditions are imposed after discretization; the full patch assignments are listed in Table 3.2.

All cases are laminar (no turbulence model), since the laminar regime is steady-converging and fully resolved on a moderate mesh, giving a clean and consistent family for the model to learn. The full dataset contains 300 cases. The sampled case parameters are summarized in Table 3.1. Using the step height as the characteristic length,

$$\mathrm{Re}_h = \frac{U_{\mathrm{inlet}}\, y_{\mathrm{mid}}}{\nu}, \tag{3.1}$$

the generated cases span $\mathrm{Re}_h \approx 67.3$ to 385.6. Each case is simulated for 3 seconds of physical time using `pimpleFoam` with adaptive time stepping (capping the Courant number at maxCo $= 5$, initial $\Delta t = 10^{-6}$ s) and a write interval of 0.01 s. The PIMPLE algorithm is configured with 3 outer correctors, 2 inner pressure correctors, and 1 non-orthogonal corrector per outer loop.

The cases are generated programmatically rather than by hand. A parametric driver draws the varied parameters with Latin hypercube sampling [37], which stratifies each range in Table 3.1 so that the 300 cases cover the parameter space evenly (the sampled distributions are shown in Figure E.1); the channel height and viscosity are held fixed across all cases. Each case is then instantiated from a templated OpenFOAM base case (one template for the transient `pimpleFoam` runs and a separate one for the steady `simpleFoam` runs used later for warm-start tests); the driver scripts the corresponding step geometry, meshes it with Gmsh, and writes the case-specific boundary conditions and run controls. All 300 cases therefore share an identical numerical setup and differ only in the sampled geometry and inlet velocity.

Table 3.1.: Parameter ranges for the 300-case data-generation set.

Quantity	Symbol	Value / range	Role
Inlet velocity	U_{inlet}	0.1004 to 0.3994 m/s	Velocity scale for Re_h
Step length	x_{mid}	0.0601 to 0.0999 m	Length of the step
Step height	y_{mid}	0.0080 to 0.0150 m	Length scale for Re_h
Channel length	$x_{\max}$	0.4 m	Fixed for all cases
Channel height	$y_{\max}$	0.03 m	Fixed for all cases
Kinematic viscosity	ν	1.5×10^{-5} m^2/s	Fixed for all cases

Figure 3.2 shows an example geometry and front-face mesh from the 300-case generation set (case 011), together with the symbolic geometry labels and boundary-patch annotations used throughout the solver setup.

The boundary conditions used for the generated transient cases are summarized in Table 3.2.

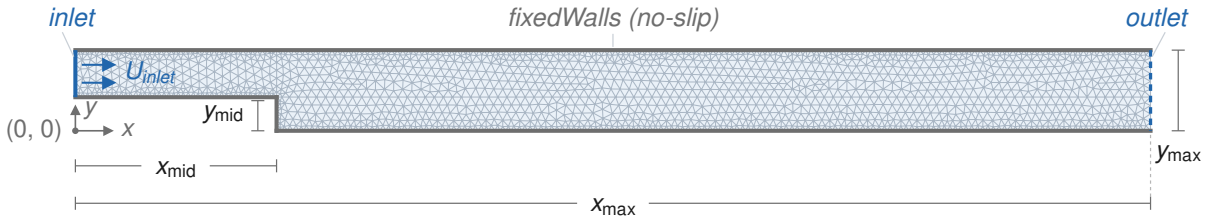


Figure 3.2.: Example case-011 mesh, geometry labels, and boundary patches.

Table 3.2.: Boundary conditions for the generated transient cases.

Patch	Velocity field `U`	Pressure field `p`	Role
`inlet`	`fixedValue (U_inlet, 0, 0)`	`zeroGradient`	inflow
`outlet`	`zeroGradient`	`fixedValue 0`	outflow
`fixedWalls`	`fixedValue (0, 0, 0)`	`zeroGradient`	no-slip walls
`front, back`	`empty`	`empty`	quasi-2D faces

The linear-solver and algorithm settings are held fixed across all cases; they define the residual tolerances against which every iteration count in Chapter 4 is measured, and are summarized in Table 3.3. The pressure equation is solved with GAMG (Gauss-Seidel smoother) and the velocity equations with a `smoothSolver` (symmetric Gauss-Seidel). Momentum convection uses a bounded first-order upwind scheme (`bounded Gauss upwind`), the remaining convective and gradient terms use linear interpolation, the Laplacian terms use `Gauss linear corrected`, and the transient term uses first-order implicit Euler integration. The steady warm-start cases use the SIMPLEC (SIMPLE-Consistent) variant of the pressure-velocity coupling with field under-relaxation, while the transient cases use the PIMPLE configuration above.

Table 3.3.: Fixed OpenFOAM solver settings for the transient data-generation cases and the steady warm-start cases.

Setting	Transient `pimpleFoam`	Steady `simpleFoam`
Coupling	PIMPLE: 3 outer, 2 pressure, 1 non-orth.	SIMPLEC, 1 non-orth.
p solver	GAMG; tol 10^{-7}, rel 0.01	GAMG; tol 10^{-8}, rel 0.1
U solver	smoothSolver; tol 10^{-7}, rel 0.01	smoothSolver; tol 10^{-8}, rel 0.1
Final correctors	p, U: tol 10^{-8}, rel 0	—
Under-relaxation	none	p 0.3, U 0.7
Stopping	outer residualControl 10^{-6}	residualControl 10^{-6}
Time stepping	adaptive; maxCo 5; init $\Delta t = 10^{-6}$ s	pseudo-time

3.2. Mesh Generation

Each case geometry is meshed using Gmsh with a coarse surface mesh profile. The 2D surface is extruded into a single layer of thickness 0.001 m in the z-direction and recombined into prism/wedge cells. The `front` and `back` patches are treated as `empty`, so the resulting volume mesh behaves as an effectively 2D problem. The mesh profile controls:

- **Coarse region size**: baseline element size in the larger channel regions (0.0034 m for the coarse profile used throughout).
- **Fine region size**: smaller element size near the selected step and channel boundary edges (0.0018 m).
- **Refinement distance bounds**: the band over which the size field blends from fine to coarse, here 0.45 mm to 4.2 mm from the refined edges.

A mesh study compared coarse and fine surface discretizations for GNN training under otherwise identical settings; the fine mesh has roughly twice as many nodes per case, with the extra nodes concentrated by the refinement bands near the step and walls. The coarse mesh reached a validation loss of 0.066 against 0.164 for the fine mesh, and the normalization scales of the two datasets are nearly identical, so the gap reflects genuinely larger per-node errors. Two effects explain it. The refinement places many of the fine mesh's nodes exactly in the high-gradient regions where prediction is hardest, so the per-node average collects more difficult points; and at a fixed message-passing depth the receptive field spans less physical distance on a finer mesh, so each prediction sees less of the flow. The coarse mesh was adopted as the default for all subsequent work.

3.3. Dataset Construction

3.3.1. Data Extraction

For each solved case, the following quantities are extracted offline from the standard OpenFOAM field output written at each saved time step:

- **Velocity field $\mathbf{u}_{xy}$**: the x and y components of velocity at each cell centre (2 values per node).
- **Pressure field p_{gauge}**: the cellwise gauge pressure defined by subtracting the spatial mean pressure of each saved snapshot (1 value per node).
- **Static geometry**: cell centre coordinates, cell volumes, face connectivity (owner/neighbour), internal face area vectors $\mathbf{S}_f$, boundary patch membership, and inlet velocity.

All data is stored in HDF5 format, with one group per trajectory (case) containing the time-indexed fields and the static mesh information. The cases are configured with a 0.01 s write interval up to 3.0 s of physical time, but some OpenFOAM runs satisfy the residual-control stopping criterion before the configured end time. The retained dataset therefore contains all available adjacent-frame transitions rather than a padded fixed length: 88,064 transitions across 300 cases. The fixed split contains 70,547 training transitions from 240 cases, 8,806 validation transitions from 30 cases, and 8,711 test transitions from 30 cases. The stored graphs have 3,159-3,417 nodes per case.

3.3.2. Surface-Prism Graph Representation

Because the mesh is a single extruded layer, the graph it defines is two-dimensional however it is read off; a graph on the volume cells and one on the `front` boundary faces are equivalent. This work reads it off the front patch, taking the nodes to be the front-face centres (the mesh drawn in Figure 3.2 is exactly this front-patch view), a construction we call the **surface-prism** representation. Through the face-owner relationship these nodes are in one-to-one correspondence with the volume cells used by the solver: each front face is owned by exactly one cell, and the single-layer extrusion guarantees that every cell owns exactly one front face. The two constructions therefore yield the same graph; the surface-prism view is a convenient way to obtain clean two-dimensional node positions and the cell-to-node mapping directly, without carrying the redundant extrusion direction.

The graph edges are derived from the internal face connectivity. For each internal face connecting cell i (owner) to cell j (neighbour), two directed edges are created: $(i \to j)$ and $(j \to i)$. Edge features for the surface-prism representation consist of three quantities:

$$\mathbf{e}_{ij} = \left[\Delta x_{ij},\ \Delta y_{ij},\ \|\Delta \mathbf{r}_{ij}\|_2\right] \tag{3.2}$$

where Δx_{ij} and Δy_{ij} are the components of the displacement vector between surface node positions, and $\|\Delta \mathbf{r}_{ij}\|_2$ is the Euclidean distance. Reverse edges carry negated displacement components but the same distance.

3.3.3. Node Features

Each graph node carries a 10-dimensional feature vector (Table 3.4). In the implementation, the dynamic velocity and pressure channels, which change at each time step, are concatenated first, followed by the static case features (boundary mask, cell volume, inlet velocity, position), which are fixed for each case. Conceptually, the channels are:

Table 3.4.: Ten-dimensional node feature vector used by the encoder.

Feature	Dimension	Description
Boundary mask	3	One-hot encoding of boundary type: inlet, outlet, wall
Cell volume	1	Normalized cell volume
Inlet velocity	1	Normalized U_{inlet} (broadcast to all nodes)
Position	2	Normalized (x, y) coordinates of the surface node
Velocity	2	Normalized (u_x, u_y) at the current time step
Pressure	1	Normalized gauge pressure at the current time step

3.3.4. Training Targets

The model is trained to predict the **one-step change** in velocity and pressure fields:

$$\hat{\mathbf{y}}_v = \left[\Delta u_x,\, \Delta u_y,\, \Delta p\right]_v = \left[u_x^{t+1} - u_x^t,\, u_y^{t+1} - u_y^t,\, p^{t+1} - p^t\right]_v \tag{3.3}$$

Predicting increments rather than absolute values is standard practice in learned simulators [4], for the reasons given in Section 2.5.

3.3.5. Normalization

The continuous input features and targets are standardized to zero mean and unit variance using statistics computed with a running mean/variance algorithm (Welford's method). In the mainline training workflow, these statistics are computed over the training split and stored in the normalization statistics file for later reuse in evaluation, TorchScript export, and solver-coupled inference. The boundary-mask channels remain as raw $0/1$ indicators. Minimum standard deviation floors are applied to prevent division by near-zero values for features with very small variance:

- Velocity targets (Δu_x, Δu_y): minimum std of 5×10^{-4}
- Pressure targets (Δp): minimum std of 1×10^{-4}
- Cell volume: minimum std of 1×10^{-11}
- Other continuous features: minimum std of 1×10^{-8}

3.4. Model Architecture

The model follows the MGN encoder-processor-decoder structure described in Section 2.5; the internal wiring of the processor blocks is shown in Figure 2.5. Figure 3.3 shows the configuration instantiated in this work: the feature dimensions, the latent

width, and the solver-facing inputs and outputs. The same network is used unchanged in training and in the solver-coupled setting; the figure annotates the latter, where the inputs are assembled from the current OpenFOAM state and the predicted increment is passed back to the solver. During training, the inputs come instead from the stored snapshots of Section 3.3 and the predicted increment enters the loss.

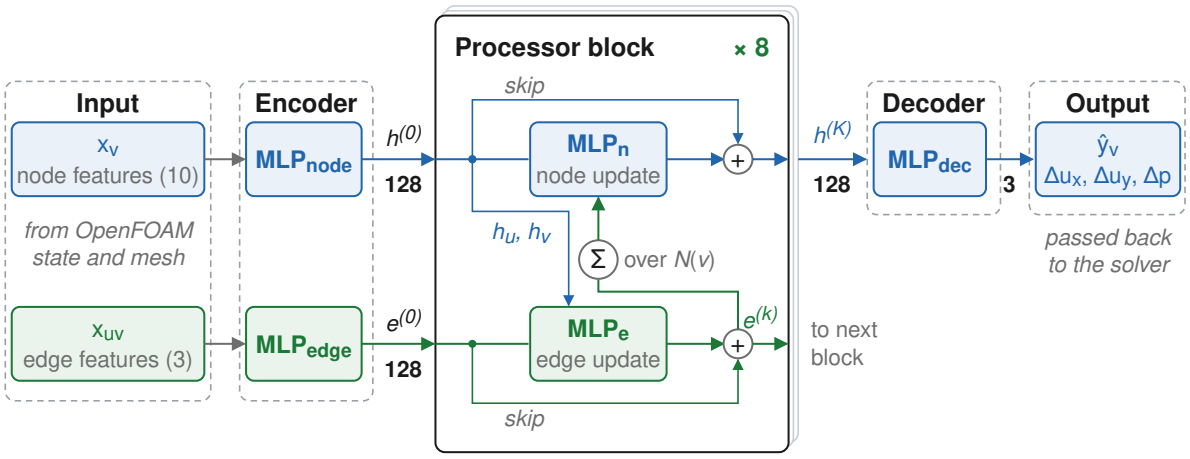


Figure 3.3.: The MGN of Figure 2.5 as instantiated in this work, with feature dimensions and solver-facing inputs and outputs.

The specific configuration used in this work is listed in Table 3.5:

Table 3.5.: Mainline full-data checkpoint hyperparameters (LayerNorm in the encoder and processor MLPs, none in the decoder).

Hyperparameter	Value
Hidden dimension	128
Message passing steps	8
MLP hidden layers	2
Aggregation	Sum
Normalization	LayerNorm
Node input dimension	10
Edge input dimension	3
Output dimension	3

With these dimensions the network has approximately 1.29 million trainable parameters, distributed across the node and edge encoders, the eight message-passing blocks, and the decoder.

This configuration was selected through a pilot architecture study (48 training cases, four hidden-dimension and message-passing combinations); the comparison is reported in Section 4.1. While the 128/12 configuration achieved the lowest pilot validation loss, the 128/8 model was selected for the mainline experiments due to its lower inference cost, an important consideration for solver-coupled deployment where the

model's forward pass is paid at every scheduled injection. The full 300-case training subsequently achieved a best validation loss of 0.007186, much lower than the pilot results due to the larger training set.

3.5. Training Procedure

3.5.1. Dataset Split

The 300 cases are divided into training (240), validation (30), and test (30) sets using a fixed stratified split, generated once and reused for every experiment. Stratifying over the case parameters keeps the three sets balanced across the flow regime; each spans almost the full Reynolds range, and the mean Re_h differs by about one percent between them (191.0, 192.8, and 193.2 for the training, validation, and test sets). The test set is held out entirely during training and hyperparameter selection, and is used only for the final solver-coupled evaluation.

3.5.2. Loss Function

The training loss is the MSE between predicted and ground-truth normalized field increments, computed over all nodes:

$$\mathcal{L} = \frac{1}{N} \sum_{v=1}^{N} \|\hat{\mathbf{y}}_v - \mathbf{y}_v\|^2 \tag{3.4}$$

where N is the number of nodes in the batch and the sum runs over all three output components (two velocity, one pressure). The loss is computed over all nodes including boundary nodes (loss mask mode `all`).

3.5.3. Optimization

Training uses the Adam optimizer [38] with the following schedule:

- **Initial learning rate**: 10^{-4}
- **Schedule**: Cosine annealing over 100 epochs to a minimum of 10^{-6}
- **Gradient clipping**: Maximum norm of 1.0
- **Weight decay**: 0 (no regularization)
- **Batch size**: 32

Training uses standard GPU training-speed optimizations (Automatic Mixed Precision (AMP) with bfloat16 precision, TF32 matrix operations, `torch.compile`, and fused Adam) on the RWTH H100 nodes; these affect only run time, not the trained model.

3.5.4. Evaluation Metrics

The trained model is assessed with two field-space metrics before any solver coupling. The **one-step error** is the per-component root-mean-square error (RMSE) between the predicted and ground-truth increments on the held-out test trajectories, evaluated one step ahead from each stored state. The **rollout error** feeds the model its own output autoregressively for a fixed number of steps and reports the velocity and pressure RMSE of the resulting fields against the stored trajectory; unless stated otherwise it is computed over the fluid-only internal nodes and reported at the final step. One-step error measures how well the model predicts a single frame; rollout error measures how far the chained predictions drift from the true trajectory over many steps, the regime that matters for warm-start initialization. The solver-facing metrics (total, pressure, and velocity linear-solver iterations, and OpenFOAM's reported execution time) are defined alongside the benchmark design in Section 3.7 and aggregated as described in Appendix D.

3.6. Solver Coupling

Coupling the model to OpenFOAM is the implementation contribution of this work, built as a reusable interface rather than a one-off script. The model hook is implemented once in a custom solver library, so a retrained model with the same inputs and outputs is deployed by replacing the exported TorchScript file (TorchScript is PyTorch's serialized model format, executable from C++ without a Python runtime; the normalization statistics are embedded at export) and editing a runtime dictionary, without recompiling the solver; changing the model's inputs or outputs additionally requires updating the export wrapper and, if needed, the C++ interface, and rebuilding. The rest of this section describes the hook placement, the field and boundary handling, the injection controls, and the PyTorch-to-C++ export path, then closes with the contrasting steady warm-start path, which needs none of this runtime machinery.

3.6.1. Custom Solver Module

The model is integrated by forking OpenFOAM's `foamRun` application, the program that drives the transient solve. OpenFOAM's modular `incompressibleFluid` solver exposes the PIMPLE stages (`prePredictor()`, `momentumPredictor()`, `pressureCorrector()`) as

methods that `foamRun` calls in a fixed sequence in each outer iteration of every time step, with no hook between them; a function object cannot intervene either, as function objects run only at time-step boundaries. To write the learned field between `prePredictor()` and `momentumPredictor()` on the first outer iteration, the implementation forks the `foamRun` time loop into a dedicated binary, `iterReduceIncompressibleFluid`, that inserts a single guarded call at exactly that point:

```
while (pimple.loop())     // PIMPLE corrector loop, inside the forked foamRun
↪  time loop
{
    solver.moveMesh();
    solver.motionCorrector();
    solver.fvModels().correct();
    solver.prePredictor();
    if (pimple.firstIter())
    {
        solver.injectPredictorFields();   // learned field injection
    }
    solver.momentumPredictor();
    solver.thermophysicalPredictor();
    solver.pressureCorrector();
    solver.postCorrector();
}
```

The `injectPredictorFields()` call is a method on a custom solver class that forks OpenFOAM's `incompressibleFluid`, and it delegates the model evaluation and field blending to a shared library, `libIterReduceCoupling`. The library loads the TorchScript-exported model once, at the first scheduled injection; each scheduled injection then follows the four steps drawn in Figure 3.4:

1. *Gather and pack fields.* The current velocity and pressure fields are read in cell order, the mean pressure is subtracted to form a gauge field, and the inlet velocity is read at run time as the mean u_x over the inlet patch; the packed tensors are copied to the GPU.
2. *TorchScript forward.* The module, which already holds the static graph tensors and normalization buffers, runs the GNN forward pass through libtorch, PyTorch's C++ library; it internally denormalizes the predicted increment, adds it to the current field, and returns absolute predicted velocity and pressure fields, which are copied back to the host.
3. *Blend and write back.* The mean pressure is re-added to the predicted gauge field, the predictions are blended with the current OpenFOAM fields, and the result is written back as the initial guess for the iterative solver.

4. *Solve the time step.* OpenFOAM's normal correction sequence runs from the injected field.

The current and predicted fields share the cell ordering, so the tensor row for cell i is the prediction for cell i; the runtime verifies that the returned tensors have one row per mesh cell before any field is written.

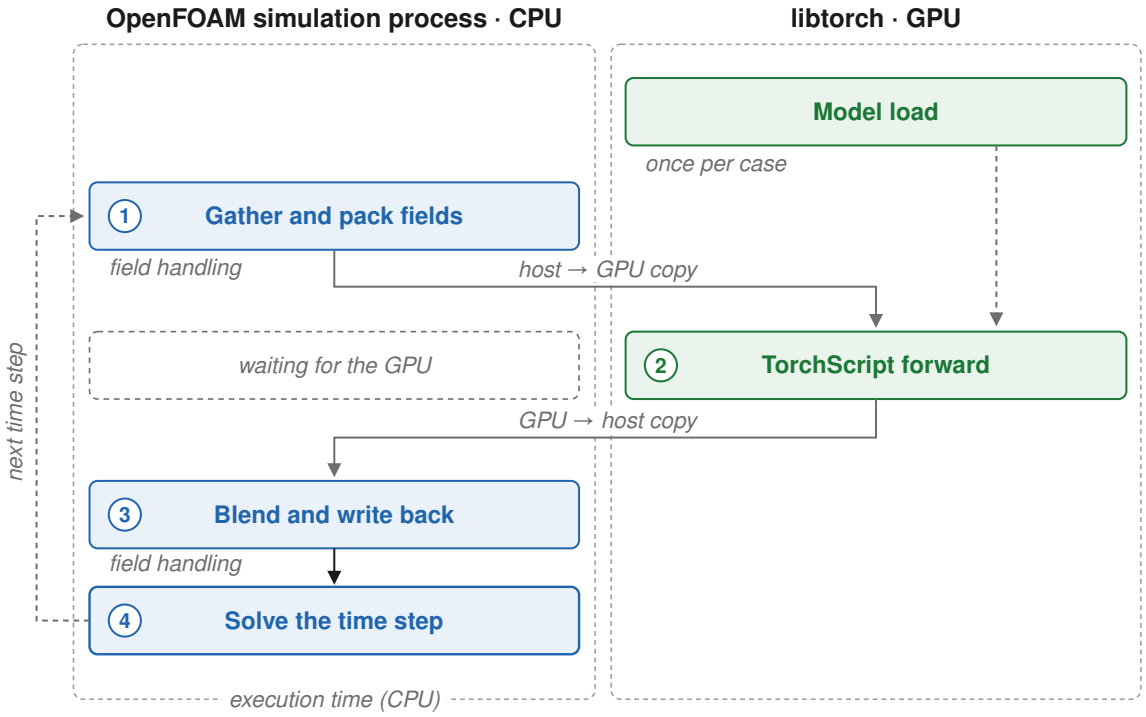


Figure 3.4.: One injected time step: the hook gathers and packs the fields, the GPU evaluates the TorchScript module while the simulation process waits, and the blended prediction is written back before the time step is solved; the module is loaded once per case.

Placing the hook early in the PIMPLE sequence, before the pressure equation is solved, means the changed field is used as the starting point for OpenFOAM's normal correction sequence rather than as a post-hoc edit. The GNN does not replace the linear solver, change the residual tolerances, or bypass the pressure correction. Limiting the call to the first outer iteration, as the loop above shows, leaves the later outer correctors to OpenFOAM and avoids re-injecting a learned field into a time step the solver has already begun correcting; the later outer iterations simply skip the guard and run the standard sequence.

Coupling a trained model to a running CFD solver spans a range of integration strategies [6] that differ mainly in whether the model runs in a separate process or inside the solver, ordered here from loosest to tightest:

- **File handoff:** the model and solver run as separate programs that exchange fields through files written between runs.
- **In-memory staging:** fields are streamed to a co-located model process at run time through an in-memory store, such as a Redis-backed orchestrator that stages data without touching the filesystem [39].
- **Embedded interpreter:** a Python interpreter is embedded in the solver and shares field arrays with it in situ, as in PythonFOAM's NumPy C-API bridge to OpenFOAM [40].
- **Compiled-in inference:** the trained network is linked directly into the compiled solver through a C/C++ library and queried in memory with no separate process, as in NNPred [41].

This work takes the last, most tightly coupled route; the exported model is loaded in-process through libtorch and called directly from the forked C++ solver, so each injection is an in-memory function call with no inter-process transfer. This keeps the per-injection overhead low enough to call the model at every time step, at the cost of a libtorch build that is binary-compatible with OpenFOAM's compiler, as detailed in the export path below.

The coupling dictionary provides runtime control over injection behaviour; the full set of configuration parameters is documented in Appendix A.

3.6.2. Boundary Treatment and Preserved Solver State

In the solver-coupled runtime, the predicted fields are blended into the current OpenFOAM cell-centred fields. When a real update is written, the usual OpenFOAM boundary-condition updates are re-applied via `correctBoundaryConditions()` for the changed fields before the solver continues. This updates patch values according to the boundary-condition types already defined in the case; it does not redefine the boundary conditions.

The current benchmark configuration preserves OpenFOAM-owned face state by default. In particular, the coupling does not refresh the face flux `phi` or the face velocity `Uf` after every field write unless optional switches kept for ablation are explicitly enabled. This was adopted after identity controls showed that refreshing `phi` or `Uf` unnecessarily increases the iteration count even when the written field is identical to the current one. With the preserve-flux defaults, both the scheduler-only `identityNoop` mode and a TorchScript identity model reproduce the cold OpenFOAM iteration counts exactly.

3.6.3. Injection Controls

The solver-coupled experiments compare fixed intervention modes rather than sweeping all available runtime parameters. The coupling dictionary nevertheless exposes the timing and blend controls needed for these modes:

- `startInjectTime`: the earliest physical time at which the model is allowed to inject.
- `injectEveryTimeStep`: whether the first-PIMPLE-iteration hook fires at every time step.
- `macroDt`: the physical-time gap between injections when `injectEveryTimeStep` is false.
- `blendMode`: whether the solver uses a fixed manual blend or a time-step-scaled blend.

The final benchmark uses these controls in two distinct ways. The repeated transient panels set `injectEveryTimeStep=true`, one run with the manual blend and one with the `dtScaled` blend; the exact step sizes and blend factors are specified in Section 3.7. The steady benchmark instead uses the offline warm-start path (Section 3.6.7), so no in-loop model calls occur during the SIMPLE solve.

This design separates two questions, namely whether repeated local transient injection helps each time step and whether a developed learned field is useful as a one-time initializer for a steady solve.

3.6.4. Transient Path and Steady Attractor

The blend rule is motivated by the trajectory argument of Section 2.4.4. Inside the transient loop the model is admitted only as a per-step correction, since a developed field would jump past the intermediate states of the trajectory; the developed-field initialization is instead tested in the steady solver, where convergence to the steady attractor is the only goal and a developed rollout is a legitimate warm-start test of whether the learned field is closer to the steady solution than the default initial condition.

Within the transient loop, the `dtScaled` rule applies only a fraction of the model correction at each step, using the blend rule defined in the next subsection with α_{eff} tied to the OpenFOAM time step. In the fixed-step benchmark this gives a controlled comparison because Δt_{OF} is known and constant; the full-update case (manual $\alpha = 1$) is the endpoint of the same rule.

3.6.5. Blend Modes

When the GNN prediction is injected into the solver's current field, the blending rule determines how the predicted field replaces or augments the existing state. Two modes are implemented:

Manual blend. The predicted field replaces the current field entirely when the user chooses $\alpha = 1$, or is linearly interpolated with a user-specified constant α:

$$\phi_{\text{injected}} = (1 - \alpha)\, \phi_{\text{current}} + \alpha\, \phi_{\text{predicted}}$$

In the results chapter, manual $\alpha = 1$ is used for the exact time-step matched one-step injection test. The steady warm start bypasses the blend machinery entirely, equivalent to a full write with $\alpha = 1$ (Section 3.6.7).

dtScaled blend. The blend factor is computed dynamically from the ratio of the solver's current time step to a user-supplied GNN reference time:

$$\alpha_{\text{eff}} = \min\left(\frac{\Delta t_{\text{OF}}}{\Delta t_{\text{GNN}}},\ \alpha_{\max}\right) \tag{3.5}$$

where Δt_{GNN} is a user-supplied reference and $\alpha_{\max}$ is a cap. In the fine-step transient experiment, $\Delta t_{\text{OF}} = 0.002$ s and $\Delta t_{\text{GNN}} = 0.01$ s, so $\alpha_{\text{eff}} = 0.2$. The experimental comparison is presented in Section 4.2.2.

3.6.6. TorchScript Export and Runtime

The trained PyTorch model is exported to TorchScript format for deployment within the C++ OpenFOAM environment, where it is loaded with `torch::jit::load()`. The runtime requires a standalone libtorch build with ABI 1 (cxx11) compatibility to match OpenFOAM's C++ toolchain. At runtime, the libtorch and CUDA runtime libraries are prepended to `LD_LIBRARY_PATH` to ensure correct symbol resolution.

The export wrapper exposes a `rollout_steps` parameter, fixed at export time rather than in the runtime dictionary; its `forward()` method loops `rollout_steps` times internally, each iteration feeding the previous prediction back as input, so a single call advances the state by `rollout_steps * dt_train`. The exported module is also case-specific; because the static graph tensors and normalization buffers are embedded, one module is exported per case mesh, and only the network weights are shared across cases. The repeated transient panels use a one-step export through the injection hook; the steady panel instead uses a longer rollout generated offline, whose final field is written as the steady solver's initial condition (Section 3.7).

Tracing a rollout wrapper through `torch.jit.trace` requires `torch.no_grad()` and `requires_grad_(False)` on wrapper parameters; without this, the trace retains every intermediate activation of the unrolled graph as part of the autograd chain, which exhausts GPU memory (approximately 89 GiB observed on an H100) for `rollout_steps >= 300`. With the no-grad wrap, the export is a one-time operation of a few seconds even at 300 steps; at run time a single one-step injection call costs of order a few milliseconds, and the measured inference costs of the configurations actually benchmarked are reported in Section 4.3.

3.6.7. Steady Warm-Start Path

The steady warm start, by contrast, needs no runtime coupling at all. An offline Python script loads the training checkpoint directly (not the TorchScript export), rolls the model out for the configured number of steps with the boundary values held fixed, and rewrites the internal fields of `0/U` and `0/p` in OpenFOAM's own text format; the solve itself then runs on the stock OpenFOAM steady solver with unmodified SIMPLE controls. The custom solver and the libtorch runtime are not involved on this path. Deploying the steady warm start therefore amounts to writing two field files, which any OpenFOAM installation can consume unchanged.

3.7. Benchmark Design

The solver-facing evaluation is split into three parts because they answer different questions.

First, sanity controls check that the custom path can be a true no-op and that iteration counts react to a deliberately poor field; the three control modes are defined with their results in Section 4.2.1.

Second, repeated transient injection is evaluated inside the custom solver on the 30-case test split. The matched run uses fixed `deltaT=0.01`, `adjustTimeStep no`, `endTime=2.0`, and one-step injection at every time step. The fine-step run uses fixed `deltaT=0.002`, `adjustTimeStep no`, `endTime=2.0`, and the dt-scaled blend with $\alpha = 0.2$. Fixed time stepping removes the adaptive time-step controller; otherwise a changed field would alter both the linear-solver iterations and the future time-step sequence.

Third, a steady-state initialization sweep evaluates the learned-rollout idea for the steady `simpleFoam` solver. Separate steady templates are used (`full_surface_coarse_v1_steady`). The cold case starts from the default initial field. For the warm case, the offline writer of Section 3.6.7 produces the initial field at rollout depth r; the standard

steady solver then runs unchanged. The sweep reports rollout depths 30 and 100 on the 30-case test split.

The reported metrics are total, pressure, and velocity linear iterations, plus the reported execution time where available; the latter is OpenFOAM's `ExecutionTime`, the CPU time of the whole simulation process. For in-loop TorchScript runs, model load, forward-pass, and total injection wall times are reported separately as wall-clock measurements. The steady `simpleFoam` execution time excludes the offline cost of producing the rollout field.

4. Results

This chapter evaluates the performance of the GNN-assisted solver. It begins with the training results (learning curves, architecture comparison, and the effect of noise injection), moves to the solver-coupled benchmarks, and closes with an interpretation of the combined results. Unless otherwise noted, all solver benchmarks use the full 300-case checkpoint with hidden dimension 128, 8 message-passing steps, and best validation loss 0.007186. The final solver evaluation uses the 30 held-out trajectories from the fixed test split.

Figure 4.1 previews what the learned model produces; starting from an undeveloped field, a 100-step GNN rollout already produces a developed field that closely resembles the converged reference. This is the qualitative basis for using the rollout as a warm-start initial field.

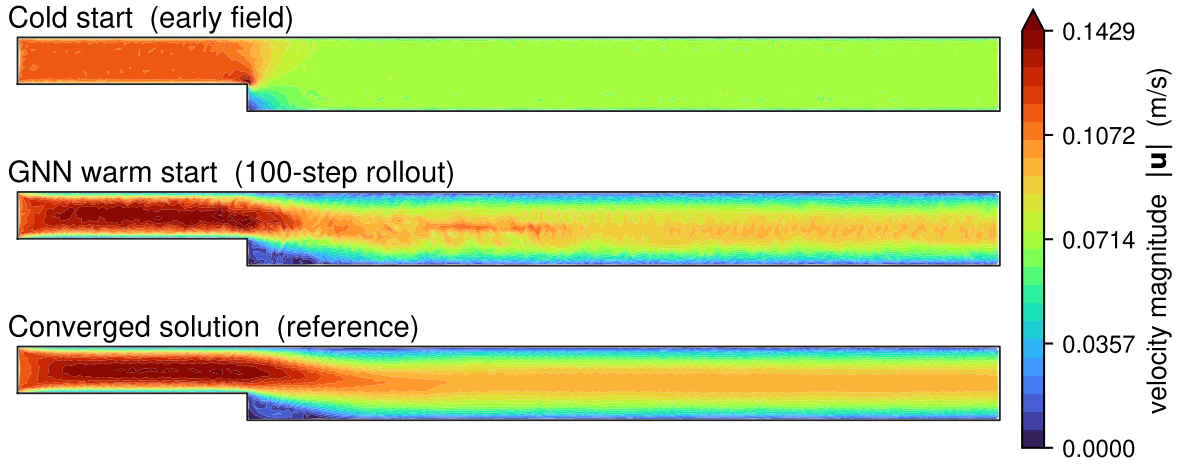


Figure 4.1.: Velocity magnitude for held-out case 188: undeveloped field, GNN warm start (100-step rollout), and converged reference.

4.1. Training Results

4.1.1. Learning Curve

The model was trained for 100 epochs on 240 training cases with validation every 5 epochs. Figure 4.2 shows the training and validation loss over the course of training.

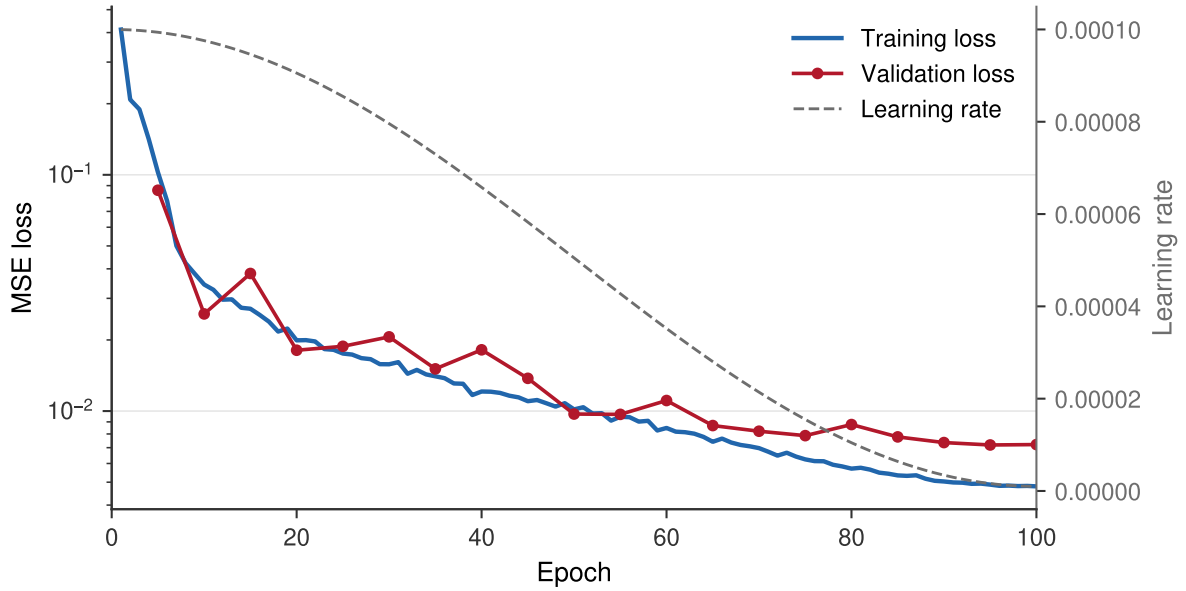


Figure 4.2.: Training and validation loss over 100 epochs.

The training loss decreased overall from 0.102 at epoch 5 to 0.005 at epoch 100, while the validation loss reached its minimum of 0.007186 at epoch 95. The training and validation curves remain close throughout the run.

The cosine annealing learning-rate schedule reduced the learning rate from 10^{-4} to 10^{-6} over the 100 epochs, with the steepest validation improvements occurring in the first 50 epochs and more gradual refinement thereafter.

4.1.2. Architecture Study

The architecture was selected through a pilot study on 64 cases (48 train, 8 validation, 8 test), comparing four configurations of hidden dimension and message-passing depth. The pilot validation losses are summarized in Table 4.1. Because the pilot trains on far less data than the full run, these values are only comparable to one another, not to the final checkpoint.

Table 4.1.: Pilot architecture-study results.

Configuration	Best pilot validation loss
96 / 8	0.05631
96 / 12	0.05146
128 / 8	0.03433
128 / 12	0.02464

Increasing the hidden dimension from 96 to 128 provided a larger improvement than increasing the message-passing depth from 8 to 12. The 128/8 configuration was

selected as a practical trade-off between prediction quality and inference cost, then trained on the full 240/30/30 split to produce the checkpoint used for every solver benchmark in this chapter (best validation loss 0.007186).

4.1.3. Training-Time Noise Injection

To reduce rollout drift, a second checkpoint was trained with MGN-style Gaussian noise injected into the input state during training [4]. The noise is per-component and scaled to 5% of each feature's natural standard deviation ($\sigma = 0.05$), with a compensating shift applied to the targets so the model learns to predict the clean delta from a noisy state. All other training settings (dataset, split, seed, and schedule) are identical to the baseline.

Table 4.2 reports one-step RMSE on the 30-case test split, while Table 4.3 reports the final-frame error after a 100-step autoregressive rollout.

Table 4.2.: One-step RMSE on the test split.

Metric	Baseline	Noise σ=0.05	Delta (%)
U_x one-step RMSE	3.87e-4	7.93e-4	+105
U_y one-step RMSE	4.35e-5	9.49e-5	+118
p one-step RMSE	8.97e-5	1.21e-4	+34

Table 4.3.: 100-step rollout RMSE on fluid-only internal nodes.

Metric (fluid-only, final step)	Baseline	Noise σ=0.05	Delta (%)
Mean velocity RMSE (100-step)	0.01163	0.00973	-16.3
Mean pressure RMSE (100-step)	0.00235	0.00189	-19.8
Noise wins (velocity)	–	22 / 30	–
Noise wins (pressure)	–	23 / 30	–

Figure 4.3 shows that the error does not simply accumulate from the first step, because two separate effects act on it. Early in the trajectory the flow is far from its developed pattern and changes quickly, so individual predictions make their largest mistakes there; but the settling dynamics are dissipative, and deviations of this kind are partly damped as both the rollout and the reference approach the same developed pattern, so the error falls at first (clearest in the pressure curves). The second effect is accumulation; each prediction starts from the previous prediction rather than from the true state, so the rollout drifts away from the reference, and this drift grows with the number of steps and eventually dominates [4]. The noise-trained model accumulates drift more slowly and ends lower at 100 steps (Table 4.3).

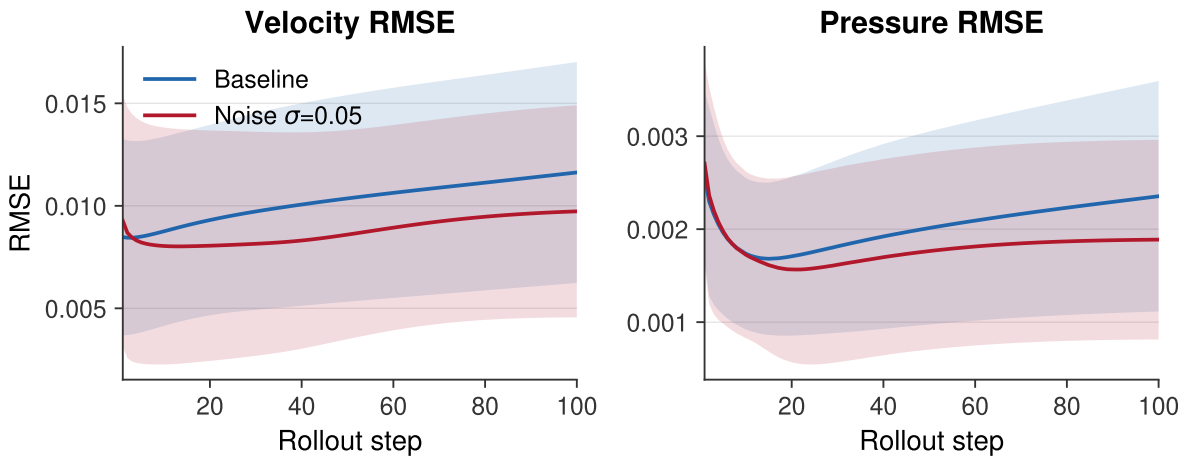


Figure 4.3.: Velocity (left) and pressure (right) RMSE against rollout step over 30 test trajectories, for the baseline and noise-trained models; shaded bands are the across-case standard deviation.

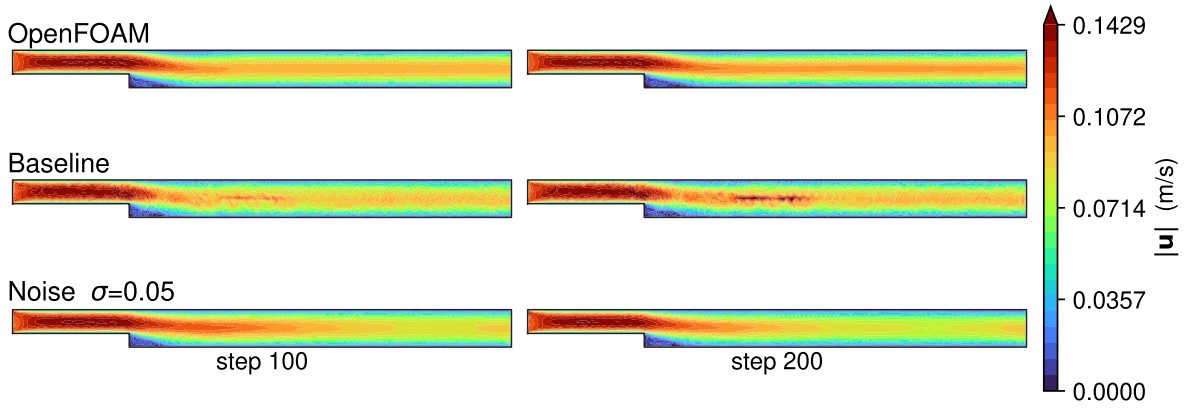


Figure 4.4.: OpenFOAM reference and baseline and noise-trained GNN rollouts (velocity magnitude) for case 188 at steps 100 and 200.

Figure 4.4 shows the same behaviour qualitatively; by step 200 the baseline rollout drifts visibly in the jet while the noise-trained rollout stays closer to the OpenFOAM reference. The combined one-step-vs-rollout picture in Table 4.2 and Table 4.3 reproduces the trade-off reported for MGN-style noise training, sacrificing one-step accuracy in exchange for a corrective signal that is activated during multi-step inference. A lower rollout error does not, however, imply a better solver warm-start. Re-running the steady `simpleFoam` sweep with the noise-trained checkpoint on the same 30 test cases *increases* total iterations by 1.76% (rollout 30) and 2.31% (rollout 100) and improves only 10 and 8 of the 30 cases, whereas the non-noise checkpoint reduces total iterations by 11.22% and 17.97% and improves all 30 (Section 4.2.3). The non-noise checkpoint is therefore used throughout.

4.2. Solver-Coupled Benchmark Results

The solver-coupled evaluation is organized into three groups. The first checks whether the custom injection machinery can be a true no-op. The second tests repeated transient injection, in two variants, one with the OpenFOAM time step matched to the model's training interval and one with a smaller OpenFOAM time step and the dt-scaled blend. The third tests the learned field as an initializer for a steady SIMPLE solve. This separation is important because repeated transient correction and steady initialization are different numerical interventions.

4.2.1. Injection-Path Sanity Controls

Before interpreting the model-assisted runs, the injection path itself was checked using three held-out cases, 011, 094, and 188 (Table 4.4). Two identity modes test whether the custom solver can reproduce the cold OpenFOAM result. `identityNoop` activates the injection scheduler but skips field writes. `identity_model` sends the current fields through the same TorchScript model-call path used by the learned predictor and returns them unchanged. A `bad_zero` mode overwrites the field with zeros and tests whether OpenFOAM iteration counts react to a deliberately uninformative guess.

Table 4.4.: Injection-path sanity controls on cases 011, 094, and 188; injection counts are totals over the three cases, the identity modes run to $t = 2.0\,\mathrm{s}$ and `bad_zero` to $t = 0.1\,\mathrm{s}$.

Mode	Injections	Total Δ%	Δp%	Fwd. wall (s)	Interpretation
`identityNoop`	600	0.00	0.00	0.000	scheduler-only no-op
`identity_model`	600	0.00	0.00	0.086	model-call path no-op
`bad_zero`	30	+13.58	+12.52	0.205	poor fields add work

The exact identity match is important. It shows that the current custom solver and TorchScript call path can be a true OpenFOAM no-op when the model output equals the input. The bad-zero result shows the opposite direction; the solver reacts measurably when the injected field is inconsistent. The remaining benchmark changes therefore come from the model's field quality, not from the injection hook itself.

4.2.2. Repeated Transient Injection

The first transient question is whether the learned one-step predictor can reduce the linear-solver work when it is inserted repeatedly into the PIMPLE loop. Two variants were tested on all 30 test cases (Table 4.5). The first exactly matches the OpenFOAM fixed time step to the training interval, using `deltaT=0.01`, `injectEveryTimeStep=true`,

and manual $\alpha = 1$. This injects the model's one-step prediction at every OpenFOAM step. The second uses a finer OpenFOAM step, `deltaT=0.002`, with `dtScaled` blending against the model training interval, giving $\alpha = 0.2$ at each injection.

Table 4.5.: Repeated transient injection on the 30-case test split (`endTime=2.0 s`, injection at every time step).

Setting	Configuration	Injections	Total Δ%	Δp%	Improved	Exec. Δ%
Matched one-step	`deltaT=0.01`, $\alpha = 1.0$	6000	+21.59	+14.80	1 / 30	+52.19
Fine-step dt-scaled	`deltaT=0.002`, $\alpha = 0.2$	30000	+18.04	+23.47	0 / 30	+37.26

Both repeated-injection variants are negative in aggregate. At the time step matched to the training interval, the model-assisted run requires 21.59% more total linear iterations and improves only one of the 30 cases. The fine-step dt-scaled version is also negative, with pressure iterations increasing by 23.47%. In the fine-step row, the velocity iteration counts are almost unchanged ($\Delta U_x = -14$, $\Delta U_y = +3$) while the pressure iterations account for essentially the whole regression ($\Delta p = +281{,}631$). This is consistent with the pressure solve being the sensitive part of the coupling.

The interpretation is not that the learned field contains no useful information. Rather, repeated one-step injection into the PIMPLE loop does not help this model; Section 4.3 examines the two mechanisms behind this.

4.2.3. Steady-State Initialization with `simpleFoam`

The steady-state benchmark tests the same initial-guess idea in a setting where there is no physical transient trajectory to preserve. The cold baseline starts from the default OpenFOAM initial field. The warm-start runs first write a GNN rollout into the `0/` directory of the steady case and then run the standard steady solver to the same SIMPLE residual controls. No model calls are made inside the steady solve.

Table 4.6.: Steady-state warm-start sweep across the 30-case test split.

Rollout	Total Δ	Total Δ%	Δp	Δp%	Cases improved	Exec. Δ%
30	-6376	-11.22	-4653	-15.24	30 / 30	-6.09
100	-10218	-17.97	-6640	-21.74	30 / 30	-11.37

The steady result is the clearest positive result (Table 4.6). A 30-step rollout already reduces total iterations by 11.22% and pressure iterations by 15.24%, and the 100-step rollout strengthens the reduction to 17.97% total and 21.74% pressure. All 30 cases

improve in both retained rollout settings. Unlike the transient in-loop runs, the steady run's execution time also decreases after the warm-start field has been written.

The steady execution times exclude the cost of producing the warm-start field, so they show a better initial condition rather than a complete wall-clock accelerator; the full accounting, including that cost, follows in Section 4.3.4.

4.2.4. Results Summary

Figure 4.5 collects the headline total- and pressure-iteration changes across the four model-assisted runs. The two repeated-injection runs sit clearly in the regression region, while the two steady warm-start runs sit in the improvement region. In three of the four runs the pressure change exceeds the total change; in the matched run pressure is the largest single contributor (56% of the added iterations).

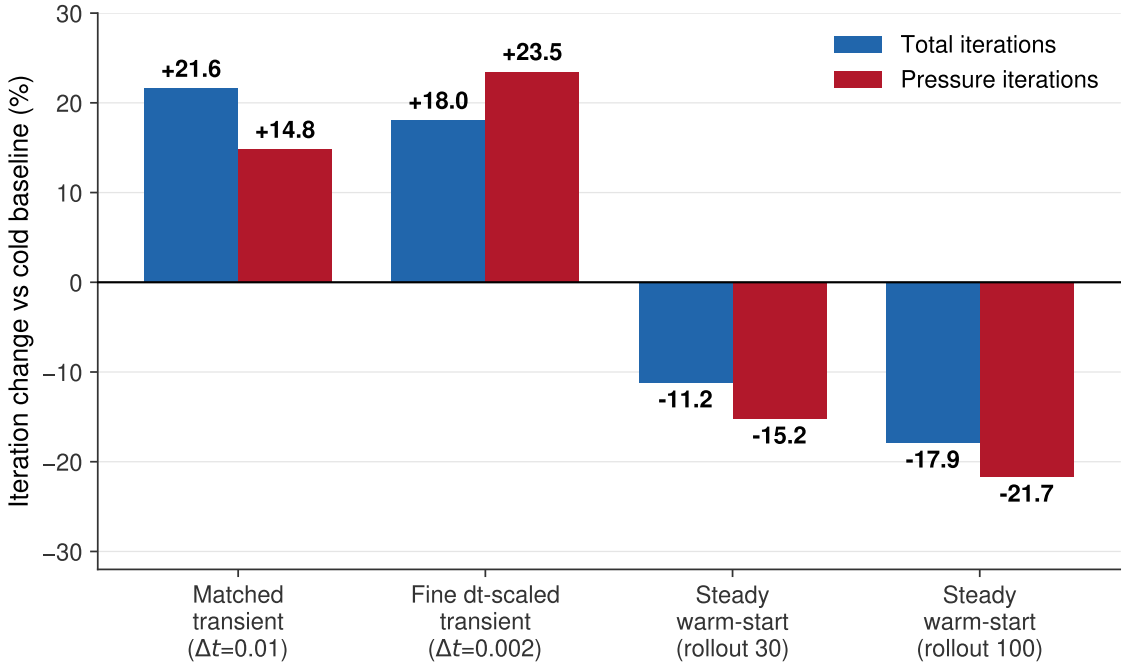


Figure 4.5.: Total- and pressure-iteration change relative to the cold baseline across the four model-assisted runs.

The headline percentages are computed by summing iteration counts across the 30 cases before taking the ratio (Appendix D), which weights each case by the work the solver actually spends on it. The per-case view is consistent with this budget view and also shows the spread. In the matched transient panel the per-case total change has a mean of $+22.2\%$ and a median of $+19.3\%$, ranging from -0.2% to $+57.2\%$; only one case improves and seven lie within 5% of the cold baseline. The fine-step panel behaves similarly (mean $+19.0\%$, median $+15.7\%$, no case improving). The steady initialization inverts this picture, improving all 30 cases at both rollout depths, with a per-case mean of -11.1% at rollout 30 and -17.9% at rollout 100, and the strongest

individual reductions reaching -27.6% and -32.8% respectively. The improvement is consistent across the whole split rather than driven by a single favourable case, which is what makes the learned field a credible initializer.

4.3. Interpretation

The benchmarks support a narrow but useful conclusion. The GNN is not a replacement for the OpenFOAM pressure solver, and the one-step model is not a useful per-step transient correction; its useful role is as a field initializer. The sanity controls (Section 4.2.1) rule out the injection hook itself as a source of the changes.

4.3.1. Pressure-Dominated Response

The strongest and most consistent changes appear in the pressure iterations. This is physically plausible, since the pressure equation is the globally coupled, elliptic part of the system (Section 2.3) and its convergence is sensitive to whether the guess already contains the correct long-range structure; the velocity equations in this setup are cheaper and leave less room for improvement.

The benchmark panels show this pressure sensitivity in both directions (Section 4.2): the transient regressions are pressure-dominated and the steady warm-start's pressure count falls fastest. The learned field is therefore most useful when it provides a developed pressure pattern before the solver starts correcting, not when it is repeatedly inserted as a local one-step perturbation.

4.3.2. Why Repeated One-Step Injection Hurts

The matched run couples the model exactly as it was trained, one prediction per 0.01 s step, and both repeated-injection variants are negative (Section 4.2.2).

Two complementary mechanisms explain why a low-error one-step field is still a poor in-loop guess for the pressure solve.

The first is a *receptive-field* limitation. Because the pressure equation couples the whole domain, as above, a good pressure guess must already carry the correct large-scale structure. A message-passing network with a fixed depth has only a K-hop receptive field, a limitation known as *under-reaching* [19]; in a single forward pass it cannot propagate information across the whole mesh, and therefore cannot represent the global pressure mode. This is the same reason single-level smoothers stall on the low-frequency pressure error and OpenFOAM solves the pressure equation with algebraic multigrid (Section 2.4), and the reason graph-based Poisson solvers adopt multi-scale

or implicit, effectively infinite-depth message passing to recover global reach [42]. The error spectra in Figure 4.6 show this directly; averaged over the held-out test set, about 90% of the one-step *pressure* error power falls in the lowest resolved wavenumber band, against 60% for u_x and 25% for u_y (spectral centroids $k_c = 1.3$, 1.9 and 3.3). The prediction error is concentrated exactly at the global scale the network cannot reach for pressure, while the velocity error, especially u_y, is comparatively more spread across smaller scales.

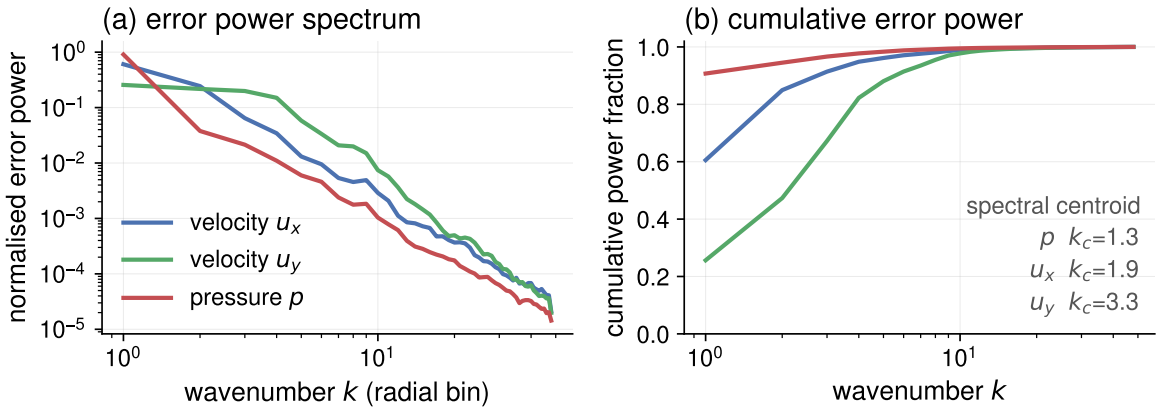


Figure 4.6.: One-step prediction-error power spectrum for u_x, u_y, p over the 30 test trajectories, normalized over wavenumbers $k \geq 1$ with the domain-mean ($k = 0$) component removed: (a) normalized spectra; (b) cumulative power fraction.

The second mechanism is *solver inconsistency*. Field RMSE measures closeness to stored solution snapshots, whereas the OpenFOAM iteration count measures how good the injected field is as a starting point for the next pressure-velocity correction under the current residual tolerances, matrix assembly, boundary treatment, and pressure-correction sequence. A field can be close in an averaged norm and still be slightly inconsistent with the continuity and pressure constraints that dominate the next linear solve, and repeating that small inconsistency at every step makes the pressure solver do more work even when the prediction looks visually plausible.

4.3.3. Initialization Is the Useful Role

Instead of correcting the field at every step, the steady benchmark uses a developed learned field once, as the initial condition of a steady solve, where no startup trajectory has to be preserved (Section 2.4.4). In that setting, the learned rollout is a legitimate initial condition; both rollout depths reduce total and pressure iterations (Section 4.2.3). The same developed field is deliberately not used as the initial condition of a transient run, where it would discard the startup trajectory the simulation is meant to compute. The training data supports this use directly, since the transient trajectories themselves settle toward the same developed pattern the steady solver targets; a sufficiently long rollout therefore lands near that pattern, which is why the 100-step rollout helps more

than the 30-step one, and why the rollout does not need to be trajectory-accurate, only closer to the converged state than the default uniform field. The repeated transient panels offer no such slack, since there the requirement is per-step consistency rather than proximity to a fixed point.

4.3.4. Wall-Clock Limitation

The method is not yet an end-to-end wall-clock accelerator. Table 4.7 breaks the measured costs down by panel, per case and cumulatively over the 30-case split. Each repeated-injection panel fires one injection per time step (200 per case at $\Delta t = 0.01$, 1000 at $\Delta t = 0.002$); the exported module is loaded once per case, every injection runs one network forward pass, and the remaining hook time is *field handling*, the data movement and blending around the inference. The steady panels pay their model cost once, offline, before the solve starts; the writer loads the PyTorch checkpoint directly (cheaper than the TorchScript module load in the transient hook) and writes the field to disk once (timed separately at about 15 ms per case), so there is no per-step field handling. The table separates two kinds of time. The execution-time rows are OpenFOAM's reported `ExecutionTime`, the CPU time the whole simulation process consumes over a run, so the cold and injected runs are directly comparable on this measure. The model load and the forward pass are waiting time instead; while the module loads from disk and the GPU evaluates the prediction, the simulation process sits idle, so these wall-clock rows lengthen the run without appearing in the CPU figure. Field handling is the one model-side cost that runs as ordinary CPU work inside the simulation process (Figure 3.4 shows where each cost sits in one injected time step), so the table splits the execution-time rise into the extra iteration work caused by the injected field and this handling share. The steady solve contains no hook and runs entirely on the CPU, so its end-to-end rows simply add the one-off generation cost onto the execution time.

Three observations follow from the table. First, in the repeated-injection panels the execution time rises ($+52.19\%$ matched, $+37.26\%$ fine-step). The extra iteration work alone is $+23.1\%$ and $+26.9\%$ of the cold run's execution time, in line with the growth in iterations, so the injected field makes the solver itself do more work and a faster model alone would not remove the penalty; the rest of the rise is the field handling inside the simulation process. Second, at the matched step the field handling is comparable to the forward pass itself, so cheaper inference alone would not remove the model-side cost either; per call, the forward pass costs 10.8 ms in the matched run and 3.7 ms in the fine-step run, where per-case startup costs amortize over five times as many calls. Third, in the steady panels the solve itself runs faster after the warm start (-6.09% at rollout 30, -11.37% at rollout 100), but the warm path does not yet pay end-to-end; adding the offline generation cost back, the warm runs are slower overall (20.7 $\rightarrow$ 25.5 s and 20.7 $\rightarrow$ 28.8 s across the split). The steady result is an iteration and solve-time

Table 4.7.: Measured costs per panel, per case and cumulative over the 30 cases; each panel separates the simulation process's CPU work from the model-side wall-clock costs outside it.

Stage	Per case	Total (30 cases)
Matched transient (200 injections/case, $\Delta t = 0.01$ s)		
CPU work in the simulation process:		
Execution time, cold → model	3.8 → 5.8 s	114.1 → 173.6 s
rise due to extra linear-solver iterations	+0.88 s	+26.36 s
rise due to field handling	+1.11 s	+33.18 s
simulation process idle, waiting on the GPU and disk (wall-clock):		
Model load (TorchScript, once per case)	1.07 s	32.04 s
GNN forward (10.8 ms/call)	2.17 s	65.06 s
Fine-step transient (1000 injections/case, $\Delta t = 0.002$ s)		
CPU work in the simulation process:		
Execution time, cold → model	12.1 → 16.6 s	363.2 → 498.5 s
rise due to extra linear-solver iterations	+3.25 s	+97.58 s
rise due to field handling	+1.26 s	+37.74 s
simulation process idle, waiting on the GPU and disk (wall-clock):		
Model load (TorchScript, once per case)	1.08 s	32.29 s
GNN forward (3.7 ms/call)	3.67 s	110.12 s
Steady warm start, rollout 30 (one offline rollout per case)		
offline generation on the GPU and disk (wall-clock):		
Model load (checkpoint, once per case)	0.06 s	1.72 s
GNN forward (30 steps per rollout)	0.13 s	3.79 s
Initial-field write (`0/U`, `0/p`)	0.02 s	0.46 s
CPU work in the simulation process:		
Execution time, cold → warm	0.69 → 0.65 s	20.7 → 19.5 s
End-to-end, cold → warm + generation	0.69 → 0.85 s	20.7 → 25.5 s
Steady warm start, rollout 100 (one offline rollout per case)		
offline generation on the GPU and disk (wall-clock):		
Model load (checkpoint, once per case)	0.06 s	1.72 s
GNN forward (100 steps per rollout)	0.27 s	8.17 s
Initial-field write (`0/U`, `0/p`)	0.02 s	0.46 s
CPU work in the simulation process:		
Execution time, cold → warm	0.69 → 0.61 s	20.7 → 18.4 s
End-to-end, cold → warm + generation	0.69 → 0.96 s	20.7 → 28.8 s

improvement, not yet an end-to-end wall-clock one; on these quickly converging cases the fixed generation cost outweighs the saving it buys (Section 5.3).

This is the practical bottleneck. The current GNN is a learned initial-guess provider, not a replacement for matrix assembly or linear solves. Each online injection adds TorchScript loading, model forwarding, field transfer, and write-back around the unchanged OpenFOAM correction sequence. Even when pressure iterations drop, the simulation still spends time on matrix assembly, boundary treatment, and data movement that the GNN does not touch, so the iteration saving does not translate one-for-one into wall-clock time; this is Amdahl's classic argument, in which the fraction of the run

time a change does not touch bounds the achievable overall speed-up [43]. Profiling and porting work on OpenFOAM reports comparable non-solver overhead in matrix assembly, residual evaluation, and host–device data movement [44].

4.3.5. Comparison with Related Work

The present approach is closest in spirit to learned initialization methods such as CFDNet [7], but it differs in two important ways. First, the model operates on an unstructured graph aligned with the OpenFOAM volume cells, rather than on a structured image-like grid. Second, the intervention point differs; CFDNet swaps its prediction in after a few warm-up iterations of the solve, whereas here only the initial field is changed before the solve starts; in both cases the solver retains full responsibility for the converged state.

The 30-case steady warm-start result is the closest direct comparison to this line of work, because the benchmark asks whether a learned field can reduce the cost of reaching a converged CFD solution. The model is trained on transient OpenFOAM trajectories and then reused as an initializer for a steady solver. The consistent reduction across all 30 cases shows that the learned pressure and velocity patterns contain information useful for the steady solve of the same geometry family, beyond the one-step transient target they were trained on.

The thesis claim is therefore narrower than "the GNN accelerates every solve"; learned fields are useful warm starts for a steady solve, while the current one-step model is not a reliable per-time-step transient correction.

5. Conclusion

5.1. Summary

This thesis coupled a trained graph neural network to OpenFOAM as a provider of starting fields and measured when that coupling lowers the iterations needed for convergence, with the solve itself left to OpenFOAM.

A learned field is an effective steady-state initial guess. Initializing a steady `simpleFoam` (SIMPLE) solve with a developed rollout field (built by applying the model repeatedly, each prediction feeding the next) reduced total linear-solver iterations by 11.2% and pressure iterations by 15.2% at a 30-step rollout, and by 18.0% and 21.7% at 100 steps, improving every one of the 30 held-out test cases. A steady solve keeps no startup trajectory, so a near-converged field is purely a better starting point for the same solution.

Used the other way, as an in-loop transient corrector, the same model is not yet useful. Blending a one-step prediction into every step of a transient `pimpleFoam` (PIMPLE) solve raised total iterations by 21.6% at a time step matched to the model's 0.01 s training interval, and by 18.0% at a finer step where only a fraction $\alpha = 0.2$ of each prediction is blended in. Two mechanisms explain this. A message-passing network with a fixed number of steps cannot propagate information across the whole domain in one pass, while the pressure field couples the domain globally, so its error concentrates at the largest spatial scales (Figure 4.6); and a field with low point-wise error need not satisfy the discrete continuity and pressure-correction equations the next solve enforces, so injecting it at every step adds correction work.

The measured changes reflect field quality, not the coupling machinery. A scheduler-only `identityNoop` mode and a TorchScript identity model (the model call path returning its input) reproduce the unassisted (cold) OpenFOAM iteration counts exactly, while a deliberately zeroed predictor inflates them, so the reported changes come from the predicted field and the blending strategy, not a faulty write path.

These outcomes use one model and one pipeline: parametric two-dimensional case generation, GNN training on 240 of 300 generated channel geometries, TorchScript export and a reusable in-process interface to a custom OpenFOAM solver for the in-loop

transient runs, and an offline field writer feeding the stock steady solver for the warm starts (Section 3.6).

The coupling itself is cheap; the remaining gap is model benefit, not interface cost. Producing the steady warm field costs about a third of a second per case (9.9 s total across the 30 test cases at rollout 100, model load and rollout included), and each in-loop injection adds a few milliseconds of inference plus a similar share of field handling (Section 4.3). The contribution is still measured in solver iterations rather than wall-clock time. Because the overhead is small and the interface is model-agnostic, a model that is both more accurate and cheap enough to generate could convert the iteration reductions into an end-to-end speed-up; establishing that is left to future work.

5.2. Practical Implications

The model has one practical deployment today, offline steady-state initialization. The learned field gives a consistent iteration reduction across the held-out split for a one-off, offline generation cost; the in-loop transient mode instead adds work at every step and is not a deployment mode for the current one-step model.

5.3. Limitations

The tested geometries are simple and quick to converge, which understates the approach; its benefit grows with the number of iterations the solve needs. On the quasi-2D channel geometries used here, the cost of producing the warm start weighs heavily against the quick solves, and what sets that ratio is the iteration count rather than the mesh size. Both the warm start and the solve scale with the number of cells, so that common factor cancels; the warm start costs a fixed number of model passes over the mesh, while the solver's iteration count climbs with Reynolds number, tighter tolerances, stronger pressure coupling, and three dimensions. The warm start's relative overhead therefore shrinks on exactly the harder, more expensive solves. The method is not tied to two dimensions either (the graph here is read from a single-layer mesh, in 3D directly from the volume cells), and 3D is the clearest favourable case, since the pressure field is hardest to solve there.

The solver evidence is also a single configuration: one geometry family, one mesh profile, and one OpenFOAM setup, with the 30 held-out cases as the full test split; different pressure solvers, tolerances, Reynolds-number ranges, and mesh resolutions may shift the response.

Field accuracy and solver benefit are not the same target. The full-data checkpoint is trained with one-step field supervision and no solver-aware term; a checkpoint trained with

rollout-stabilizing noise reaches a lower 100-step rollout error yet makes a worse steady warm start (Section 4.1), so lower field error does not guarantee fewer solver iterations, a decoupling also documented for learned warm starts in fixed-point optimization [45].

Neither coupling mode is faster end-to-end today, and the deficit has a different cause in each. In the repeated transient runs the deficit is not an overhead problem; the injected field itself increases the iteration count, so the solver does more work before any model cost is added, and cheaper inference would not change the sign. In the steady runs the warm start does its job, with fewer iterations and a faster solve, and the deficit comes only from the one-off cost of generating the field, which outweighs the saving on these quick-converging cases. The implementation is additionally a prototype, built to measure the coupling rather than to optimize it, so the measured model-side costs are an upper bound; routine engineering could trim them, narrowing the steady end-to-end gap, though it would not change the transient result, where the injected field itself adds iterations.

5.4. Future Work

Training for solver response rather than field error is the most promising direction, and the hardest. The current model minimizes one-step field loss, which the results show is not the same as minimizing solver iterations. A solver-aware objective would instead reward predictions that reduce pressure iterations after blending; the difficulty is the training signal, since how a written field changes the downstream solve is expensive to obtain and not directly differentiable through OpenFOAM. Both halves of that problem now have published templates: unrolling solver iterations behind the network and differentiating through them trains warm starts [45] and in-loop corrections [8], and a differentiable PISO finite-volume solver now exists as peer-reviewed open-source code [46], so a differentiable training stand-in for the OpenFOAM solve is no longer hypothetical. Learning the preconditioner inside the linear solve, as in recent learned conjugate-gradient preconditioners [47], is a complementary route, since field-level initialization and learned preconditioning reduce different parts of the linear-solver cost; recent variants trained on residual losses are reported to run faster end-to-end than classical preconditioners on pressure-Poisson systems [48].

The steady-state result should also be tested beyond one setting. A larger benchmark spanning mesh resolutions, pressure solvers, and tolerances would show how much of the pressure-iteration reduction is general, and a harder-converging solve (higher Reynolds number, tighter tolerances, or fully 3D) is the natural next test, where the scaling argument above favours the approach.

Lowering the one-off generation cost is the remaining wall-clock lever on cases this small. That cost scales with the number of model passes in the rollout; distillation, fewer message-passing steps, or a smaller model trained for initialization would lower the break-even point, complementing the harder-case route above.

References

[1] J. H. Ferziger, M. Perić, and R. L. Street, *Computational methods for fluid dynamics*, 4th ed. Springer, 2019. doi: 10.1007/978-3-319-99693-6.

[2] F. Moukalled, L. Mangani, and M. Darwish, *The finite volume method in computational fluid dynamics: An advanced introduction with OpenFOAM and Matlab*. Springer, 2016. doi: 10.1007/978-3-319-16874-6.

[3] Y. Saad, *Iterative methods for sparse linear systems*, 2nd ed. SIAM, 2003. doi: 10.1137/1.9780898718003.

[4] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, "Learning mesh-based simulation with graph networks," in *International conference on learning representations*, 2021.

[5] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia, "Learning to simulate complex physics with graph networks," in *International conference on machine learning*, PMLR, 2020, pp. 8459–8468.

[6] R. Vinuesa and S. L. Brunton, "Enhancing computational fluid dynamics with machine learning," *Nature Computational Science*, vol. 2, no. 6, pp. 358–366, 2022, doi: 10.1038/s43588-022-00264-7.

[7] O. Obiols-Sales, A. Vishnu, N. Malaya, and A. Chandramowlishwaran, "CFDNet: A deep learning-based accelerator for fluid simulations," in *Proceedings of the 34th ACM international conference on supercomputing (ICS '20)*, Association for Computing Machinery, 2020, pp. 1–12. doi: 10.1145/3392717.3392772.

[8] K. Um, R. Brand, Y. R. Fei, P. Holl, and N. Thuerey, "Solver-in-the-loop: Learning from differentiable physics to interact with iterative PDE-solvers," in *Advances in neural information processing systems*, 2020, pp. 6111–6122. Available: https://proceedings.neurips.cc/paper/2020/hash/43e4e6a6f341e00671e123714de019a8-Abstract.html

[9] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021, doi: 10.1109/TNNLS.2020.2978386.

[10] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Computers in Physics*, vol. 12, no. 6, pp. 620–631, 1998, doi: 10.1063/1.168744.

[11] S. V. Patankar and D. B. Spalding, "A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows," *International Journal of Heat and Mass Transfer*, vol. 15, no. 10, pp. 1787–1806, 1972.

[12] R. I. Issa, "Solution of the implicitly discretised fluid flow equations by operator-splitting," *Journal of Computational Physics*, vol. 62, no. 1, pp. 40–65, 1986.

[13] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*. Academic Press, 2001.

[14] K. Stüben, "A review of algebraic multigrid," *Journal of Computational and Applied Mathematics*, vol. 128, no. 1–2, pp. 281–309, 2001, doi: 10.1016/S0377-0427(00)00516-1.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International conference on learning representations*, 2017.

[16] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.

[17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International conference on learning representations*, 2018.

[18] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International conference on machine learning*, PMLR, 2017, pp. 1263–1272.

[19] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," in *International conference on learning representations*, 2021.

[20] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International conference on learning representations*, 2019.

[21] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016, Available: https://arxiv.org/abs/1607.06450

[22] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the AAAI conference on artificial intelligence*, 2018.

[23] J. Brandstetter, D. E. Worrall, and M. Welling, "Message passing neural PDE solvers," in *International conference on learning representations*, 2022.

[24] Z. Li *et al.*, "Fourier neural operator for parametric partial differential equations," in *International conference on learning representations*, 2021.

[25] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.

[26] B. Alkin *et al.*, "AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers," *Transactions on Machine Learning Research*, 2025, Available: https://openreview.net/forum?id=nwQ8nitlTZ

[27] X.-H. Zhou *et al.*, "Neural operator-based super-fidelity: A warm-start approach for accelerating steady-state simulations," *Journal of Computational Physics*, vol. 529, p. 113871, 2025, doi: 10.1016/j.jcp.2025.113871.

[28] C. Caron, P. Lauret, and A. Bastide, "Coupling machine learning local predictions with a computational fluid dynamics solver to accelerate transient buoyant plume simulations," in *Proceedings of the twelfth international conference on computational fluid dynamics (ICCFD12), kobe, japan*, 2024. Available: https://arxiv.org/abs/2409.07175

[29] P. Sousa, A. Afonso, and C. V. Rodrigues, "Application of machine learning to model the pressure poisson equation for fluid flow on generic geometries," *Neural Computing and Applications*, vol. 36, pp. 16581–16606, 2024, doi: 10.1007/s00521-024-09935-0.

[30] J. Tompson, K. Schlachter, P. Sprechmann, and K. Perlin, "Accelerating eulerian fluid simulation with convolutional networks," in *International conference on machine learning*, PMLR, 2017, pp. 3424–3433.

[31] E. Ajuria Illarramendi, A. Alguacil, M. Bauerheim, A. Misdariis, B. Cuenot, and E. Benazera, "Towards an hybrid computational strategy based on deep learning for incompressible flows," in *AIAA aviation 2020 forum*, 2020. doi: 10.2514/6.2020-3058.

[32] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, p. e2101784118, 2021.

[33] J.-T. Hsieh, S. Zhao, S. Eismann, L. Mirabella, and S. Ermon, "Learning neural PDE solvers with convergence guarantees," in *International conference on learning representations*, 2019.

[34] J. Chen, "Graph neural preconditioners for iterative solutions of sparse linear systems," in *International conference on learning representations*, 2025. Available: https://arxiv.org/abs/2406.00809

[35] I. Luz, M. Galun, H. Maron, R. Basri, and I. Yavneh, "Learning algebraic multigrid using graph neural networks," in *International conference on machine learning*, PMLR, 2020, pp. 6489–6499.

[36] D. Greenfeld, M. Galun, R. Kimmel, I. Yavneh, and R. Basri, "Learning to optimize multigrid PDE solvers," in *International conference on machine learning*, PMLR, 2019.

[37] M. D. McKay, R. J. Beckman, and W. J. Conover, "Comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979, doi: 10.1080/00401706.1979.10489755.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International conference on learning representations*, 2015.

[39] S. Partee *et al.*, "Using machine learning at scale in numerical simulations with SmartSim: An application to ocean climate modeling," *Journal of Computational Science*, vol. 62, p. 101707, 2022, doi: 10.1016/j.jocs.2022.101707.

[40] R. Maulik, D. K. Fytanidis, B. Lusch, V. Vishwanath, and S. Patel, "PythonFOAM: In-situ data analyses with OpenFOAM and Python," *Journal of Computational Science*, vol. 62, p. 101750, 2022, doi: 10.1016/j.jocs.2022.101750.

[41] W. Liu, Z. Song, and J. Fang, "NNPred: Deploying neural networks in computational fluid dynamics codes to facilitate data-driven modeling studies," *Computer Physics Communications*, vol. 290, p. 108775, 2023, doi: 10.1016/j.cpc.2023.108775.

[42] M. Nastorg, M. A. Bucci, T. Faney, J.-M. Gratien, G. Charpiat, and M. Schoenauer, "An implicit GNN solver for Poisson-like problems," *Computers & Mathematics with Applications*, vol. 176, pp. 270–288, 2024, doi: 10.1016/j.camwa.2024.10.036.

[43] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the april 18-20, 1967, spring joint computer conference*, Association for Computing Machinery, 1967, pp. 483–485. doi: 10.1145/1465482.1465560.

[44] F. Piscaglia and F. Ghioldi, "GPU acceleration of CFD simulations in OpenFOAM," *Aerospace*, vol. 10, no. 9, p. 792, 2023, doi: 10.3390/aerospace10090792.

[45] R. Sambharya, G. Hall, B. Amos, and B. Stellato, "Learning to warm-start fixed-point optimization algorithms," *Journal of Machine Learning Research*, vol. 25, no. 166, pp. 1–46, 2024.

[46] A. Franz, H. Wei, L. Guastoni, and N. Thuerey, "PICT – a differentiable, GPU-accelerated multi-block PISO solver for simulation-coupled learning tasks in fluid dynamics," *Journal of Computational Physics*, vol. 544, p. 114433, 2026, doi: 10.1016/j.jcp.2025.114433.

[47] Y. Li, P. Y. Chen, T. Du, and W. Matusik, "Learning preconditioners for conjugate gradient PDE solvers," in *Proceedings of the 40th international conference on machine learning*, in Proceedings of machine learning research, vol. 202. PMLR, 2023, pp. 19425–19439.

[48] K. W. Lan, E. Gueidon, A. Kaneda, J. Panetta, and J. Teran, "A neural-preconditioned Poisson solver for mixed Dirichlet and Neumann boundary conditions," in *Proceedings of the 41st international conference on machine learning*, in Proceedings of machine learning research, vol. 235. PMLR, 2024, pp. 25976–25994.

A. Solver Coupling Configuration

The runtime behaviour of the GNN field injection is controlled through the `iterReduceCouplingDict` dictionary placed in the `system/` directory of each OpenFOAM case. The exact values differ between repeated transient injection, steady initialization, and sanity controls, but the current benchmark configuration uses the same core fields:

```
enabled          true;
modelPath        "constant/iterreduce/model_inference.ts";
contextDir       "constant/iterreduce/context";
device           "cuda:0";

predictorMode    "model";
blendMode        "dtScaled";
guessBlendAlpha  1.0;
gnnTrainingDt    0.01;
dtScaledAlphaMax 1.0;

startInjectTime  0.0;
macroDt          0.01;
injectEveryTimeStep true;

applyVelocityGuess true;
applyPressureGuess true;
correctBoundariesAfterInjection true;
refreshPhiAfterInjection false;
refreshUfAfterInjection false;
preserveUz       true;
```

The key parameters are:

- **`enabled`**: master switch for the injection mechanism.
- **`predictorMode`**: selects between the trained model (`"model"`) and the identity controls (`"identity"`, `"identityNoop"`). Synthetic baselines such as the zero-field predictor are realized as `"model"` with a synthetic TorchScript module swapped in by the benchmark harness, not as a dictionary value.

- **blendMode**: controls how the prediction is combined with the current field. "manual" uses a fixed blend factor guessBlendAlpha; "dtScaled" computes the factor from the ratio of the solver's current time step to gnnTrainingDt.
- **injectEveryTimeStep** and **macroDt**: control whether the first-PIMPLE-iteration hook fires every time step or at a coarser physical interval.
- **correctBoundariesAfterInjection**: applies OpenFOAM's existing boundary-condition update to fields that were actually changed.
- **refreshPhiAfterInjection** and **refreshUfAfterInjection**: legacy ablation switches. They are kept false in the current thesis benchmarks so OpenFOAM-owned face state is preserved and the pressure corrector updates fluxes naturally.

The modelPath points to the TorchScript-exported model. In the current exporter, the static graph tensors and normalization buffers are embedded in the TorchScript module, so the C++ runtime supplies only the current OpenFOAM fields and scalar inlet velocity at inference time; contextDir must still exist on disk (it is checked at model load) although its contents are not read by the current exporter.

B. Current Benchmark Evidence

The thesis uses a single consolidated 30-case benchmark run executed after the preserve-flux solver cleanup. Older exploratory and pre-cleanup panels are historical diagnostics and are not used as the main thesis evidence. Table B.1 maps the current claims to their stored artifacts. The artifact names are shown relative to their benchmark campaign roots to keep the printed table readable.

Table B.1.: Current benchmark evidence used for the thesis solver claims.

Panel	Configuration	Total Δ%	Δp%	Improved	Artifact
Sanity, identity	cases 011/094/188	0.00	0.00	0/3	`sanity/`
Sanity, `bad_zero`	cases 011/094/188	+13.58	+12.52	0/3	`sanity/`
Matched transient	`deltaT=0.01`, $\alpha = 1$	+21.59	+14.80	1/30	`transient_matched/`
Fine-step transient	`deltaT=0.002`, $\alpha = 0.2$	+18.04	+23.47	0/30	`transient_finestep/`
Steady, r=30	rollout 30	-11.22	-15.24	30/30	`steady/`
Steady, r=100	rollout 100	-17.97	-21.74	30/30	(same)

The same sign convention is used in every table. Negative iteration changes mean that the model-assisted or warm-start run required fewer iterations than the cold baseline. Positive values are regressions.

All artifact keys map to subdirectories of the single consolidated benchmark run `timing_full_30case_20260611T103707Z`, which contains every panel of the final evaluation: the sanity controls, both repeated transient panels, and the steady warm-start sweep, with the headline numbers in the run's `consolidated_timing.json` and the per-case records in each subdirectory.

C. Reproducibility and Artifact Map

The main thesis artifacts are summarized in Table C.1. The table uses short identifiers so that the printed thesis remains readable; the repository runbook stores the exact commands and paths.

Table C.1.: Main artifacts used by the thesis pipeline.

Item	Short identifier	Role
Dataset	full surface HDF5	300-case transient training set
Split	fixed 240/30/30 split	train/validation/test partition
Main checkpoint	full-data hd128 run	state-only model used for solver benchmarks
Noise checkpoint	noise 0.05 comparison	rollout-stability comparison model
Runtime solver	custom OpenFOAM solver	`iterReduceIncompressibleFluid` with injection hook
Coupling library	libtorch coupling library	`libIterReduceCoupling` for TorchScript model loading and field blending
Current benchmark root	consolidated 30-case run	solver evidence for the final thesis story

The fixed split file is important because the solver benchmarks use held-out cases rather than training cases. The final solver panels use the 30 trajectories under the `test` key in `full_split_240_30_30_v1.json`, stored in `gnn_preconditioner/manifests/`. Older pilot and exploratory panels remain useful for development history but are not the primary evidence for the final claims.

Environment and Build Checks

The results in this thesis were produced on NVIDIA H100 GPUs, with the fixed seed 1337 used for all model training and the dataset split. The main software dependencies are listed in Table C.2.

Table C.2.: Main software dependencies used to produce the thesis results.

Component	Version	Role
OpenFOAM	12	baseline and custom CFD solver
libtorch	2.5.1 (cu121, cxx11 ABI)	C++ Torch runtime linked by the custom solver
PyTorch	2.5.1 (CUDA 12.1)	model training and TorchScript export
PyTorch Geometric	2.7.0	graph batching and data utilities
h5py	3.12.1	dataset storage (HDF5)
NumPy	1.26.4	array handling and case sampling
Gmsh	4.13	case mesh generation
Quarto / TeX Live	1.8 / 2024	thesis document build

All OpenFOAM, TorchScript, and benchmark commands require the project runtime environment to be loaded in the same shell invocation:

```
source gnn_preconditioner/setup_env.sh --runtime-only
```

The minimal smoke check used before solver work is:

```
source gnn_preconditioner/setup_env.sh --runtime-only
which pimpleFoam
which iterReduceIncompressibleFluid
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

The expected result is that `pimpleFoam` resolves to the OpenFOAM 12 installation, `iterReduceIncompressibleFluid` resolves to the user-built custom solver, and PyTorch reports CUDA availability on a GPU node. For CPU-only login or development sessions, thesis rendering and text work are still possible, but GPU-coupled benchmarks should not be launched.

The custom solver build has three relevant targets:

Table C.3.: OpenFOAM build targets required for solver-coupled inference.

Target	Purpose	Expected location
`libIterReduceCoupling`	loads TorchScript model and blends fields	user OpenFOAM lib dir
solver module	extends `incompressibleFluid`	user OpenFOAM lib dir
solver app	exposes `iterReduceIncompressibleFluid`	user OpenFOAM app dir

The custom artifacts are built into user space and never modify the stock installation. Two helper scripts automate this: `sync_to_user_dir.sh` copies the sources into the OpenFOAM user directory, and `build_user_dir.sh` compiles them with OpenFOAM's

`wmake`, installing the coupling library and solver module into `FOAM_USER_LIBBIN` and the `iterReduceIncompressibleFluid` application into `FOAM_USER_APPBIN`. At run time, the standalone cxx11 libtorch and the CUDA runtime are prepended to `LD_LIBRARY_PATH` so that symbols resolve against the same ABI the solver was compiled with. The baseline cold runs use stock OpenFOAM (`foamRun -solver incompressibleFluid`); the model-coupled runs select the custom solver. The exact build, training, export, and benchmark commands are kept in the repository runbook rather than reproduced here.

D. Aggregation Rules

For each benchmark row, the signed total-iteration change is computed as

$$\Delta N = N_{\text{model}} - N_{\text{cold}},$$

and the percentage change is

$$\Delta N_{\%} = 100 \frac{N_{\text{model}} - N_{\text{cold}}}{N_{\text{cold}}}.$$

The same definition is used for pressure iterations, velocity iterations, total iterations, and the reported execution time. Negative values are improvements because the model-assisted or warm-start run required fewer iterations or less time. Positive values are regressions and are reported as such.

For aggregate tables, the cold counts are summed across the 30 cases before the percentage is computed. This is different from taking the arithmetic mean of the 30 individual percentages. The sum-based aggregation is used because it weights each case by the number of iterations actually spent by the solver. A case with a larger pressure-iteration budget should influence the total iteration budget more than a case that already converges cheaply.

The case-by-case view is still retained as a robustness check. The results tables report how many individual cases improved, and the raw benchmark JSON/CSV files store the per-case deltas, means, medians, and best/worst cases. The headline percentage therefore answers the solver-budget question, while the case counts answer the generality question.

The same sign convention is used in every results table:

Table D.1.: Sign convention used in all results tables.

Quantity	Negative value means	Positive value means
Total Δ	fewer total iterations	more total iterations
Δp	fewer pressure iterations	more pressure iterations
Time Δ	lower execution time	higher execution time
Forward wall	model-call cost only	not a speed-up metric

E. Case Parameter Distributions

Figure E.1 shows the marginal distributions of the Latin-hypercube-sampled parameters across the 300-case family. Each marginal is close to uniform over its range in Table 3.1, with the 20 bins holding 13–17 cases each against the 15 expected for exact uniformity, the stratified coverage the sampler is designed to produce.

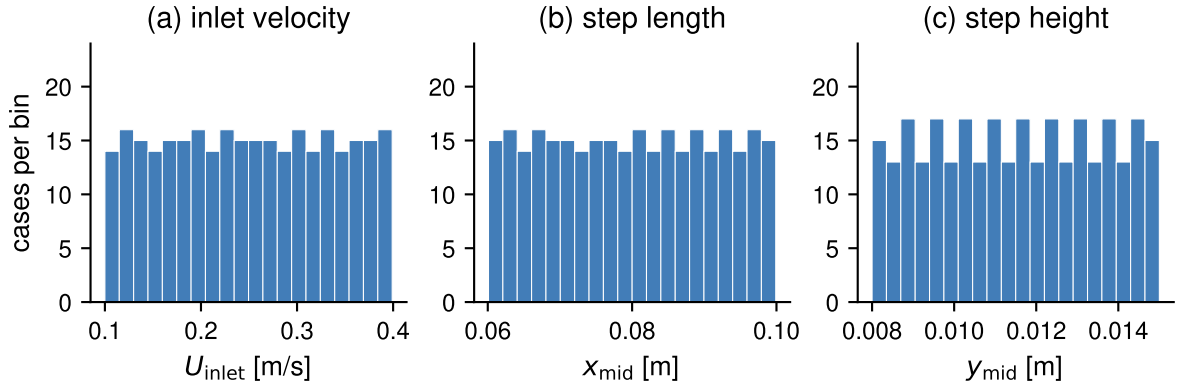


Figure E.1.: Sampled case-parameter distributions across the 300-case family.